Designing A Smart Microgrid Emulation

Summary: This project develops a smart microgrid emulation framework for household energy management, with the aim of reducing household energy costs and reliance on the main grid. The motivation for this project comes from rising energy demand, increasing energy costs and wider adoption of distributed energy resources such as rooftop photovoltaic generation (PV) and battery storage. As a result, households are increasingly able to act as prosumers instead of simple energy consumers, by generating, storing, consuming and potentially trading energy in a local community market

To achieve this aim, a software emulation framework was developed to represent the key functions required for a local peer-to-peer (P2P) energy system. At the core of the framework are the household agents, which are responsible for simulating household demand, PV generation, battery operation and trading decisions. A short-term forecasting function lies within the household agent which forecasts future demand to support proactive energy trading decisions. Based on the outputs of households, a market operator performs P2P trading between households, while a Balancing operator settles unmatched results and remaining deviations. Finally, outcomes are stored and presented on a dashboard, including trading outcomes and estimated cost savings.

The project shows that household forecasting, local P2P trading and balancing can work together to reduce household energy costs, encourage local energy sharing, and reduce reliance on the main grid.



Supervised by Dr Rasheed Hussain

School of Electrical, Electronic, and Mechanical Engineering

University of Bristol

2025/2026

Declaration

This project report is submitted towards an application for a degree at the University of Bristol. The report is based upon independent work by the candidates. All contributions from others have been acknowledged and the supervisor is identified on the front page. The views expressed within the report are those of the authors and not of the University of Bristol.

We hereby assert our right to be identified as the authors of this report. We give permission to the University of Bristol Library to add this report to its stock and to make it available for consultation in the library, and for inter-library lending for use in another library. It may be copied in full or in part for any bona fide library or research worker on the understanding that users are made aware of their obligations under copyright legislation.

We hereby declare that the above statements are true.



© Copyright, {Jake Evans, George Huang, Jin Lan, Yujia Dai, Gongxi Yao 2026}

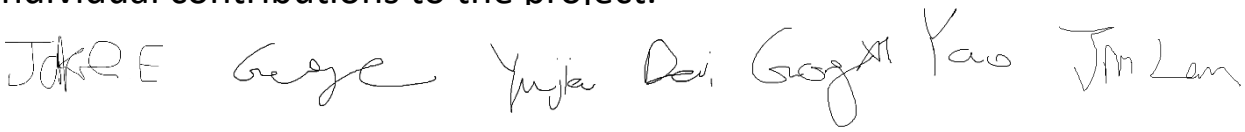
Certification of ownership of the copyright in a dissertation presented as part of and in accordance with the requirements for the relevant degree at the University of Bristol.

This report is the property of the University of Bristol Library and may only be used with due regard to the authors. Bibliographical references may be noted but no part may be copied for use or quotation in any published work without prior permission of the authors. In addition, due acknowledgement for any use must be made.

Work Allocation

Jake Evans	Forecasting Module
George Huang	Household agent
Gongxi Yao	Market Operator
Yujia Dai	Balancing operator
Jin Lan	Project Integration and Deployment

We confirm that the information on this page accurately describes our individual contributions to the project.



1 Introduction (JE, GH, JL, YD, GY) 5
2 System architecture (JE, GH, JL, YD, GY) 6
3 Forecasting model (JE) 7
3.1 Motivation 7
3.2 Problem formulation, Dataset and Preprocessing 7
3.3. Experimental Design and Evaluation Methodology 9
3.4. Core Results and Model Selection 10
3.5. Extension Studies 12
3.6 Conclusion 14
4 Household Agents (GH) 15
4.1 Planning and design process 15
4.2 Battery modelling 18
4.3 PV generation model 20
4.4 Agent model 21
4.5 Forecast processing 21
5 Market Operator (GY) 23
5.1 Market design and related work 23
5.2 Overview 23
5.3 Order formation and order book 24
5.4 Market Mechanisms 24
5.5 Bid and ask strategy 26
5.6 Market Outputs 28

5.7 Comparison 28
6 Balancing Operator (YD) 30
6.1 Overview 30
6.2 Stage 1: Unmatched Order Settlement 31
6.3 Stage 2: Deviation Settlement 31
6.4 Internal Matching 33
6.5 Implementation and Comparison 34
7 Project Integration and Deployment (JL) 38
7.1 Introduction 38
7.2 Execution foundation 39
7.3 Docker Compose as the System assembly layer 39
7.4 Tailscale as the secure Multi-Host Deployment Layer 40
7.5 Inter-Service Communication Layer 41
7.6 Central control and timing layer 42
7.7 Persistent data storage layer 44
7.8 Monitoring and visualisation layer 44
8. Results (JE, GH, JL) 45
8.1 Forecasting model used in the simulations 45
8.2 Seasonal and full-year results for 30 households 45
8.3 Pricing behaviour and performance for the 100-household community 46
9 Limitations and Further Research (Shared) 47
9.1 Forecasting module 47
9.2 Household agent 47
9.3 Market Operator 48
9.4 Balancing Operator 48
9.5 Visualisation and Deployment Framework 48
10 Conclusion 49
References 50

1 Introduction (JE, GH, JL, YD, GY)

Increasing residential energy use, together with the growing relevance of home energy management systems, has increased interest in smarter household-level energy systems [1]. As more households adopt distributed energy resources (DERs), particularly photovoltaic (PV) generation and battery storage, they can operate as prosumers, capable of generating, storing, consuming and potentially trading electricity within a local community energy market [2]. This creates opportunities to reduce energy cost [3], improve environmental sustainability and reduce grid reliance through better coordination of local energy resources and peer-to-peer (P2P) trading [2].

1.1 Background

Smart microgrids and local energy markets have been increasingly studied as approaches for improving the use of distributed energy resources at the household and community level. In these systems, electricity is no longer distributed only from a centralised generation source. Instead, households acting as prosumers can produce and store energy locally through PV systems and battery storage. With P2P trading included, surplus or deficit energy can be exchanged through a local market. [3].

Since distributed energy systems require coordinating multiple participants at the same time due to their decentralised nature, they introduce additional coordination challenges [2], making the system more complex as opposed to traditional centralised grid energy systems. Home energy management systems are therefore needed to coordinate local resources, household demand and operational control more effectively [1]. In this emulation, the coordination challenge is represented through interactions between household demand, local generation, battery operation, market participation, balancing and grid exchange.

1.2 Objectives

The overall objective of this project is to develop an emulation framework for a smart microgrid and local energy market system, supporting realistic household behaviour, local trading, and system coordination.

More specifically, the project aims to:

1) Develop a household-level forecasting function, so energy decisions can account for future conditions rather than relying solely on current conditions
2) Model realistic household agents, incorporating demand, PV generation, battery storage and grid interaction
3) Develop a local energy market to enable internal P2P trading based on the forecasted household energy surplus and deficit, support reductions in the local energy cost and grid reliance and generate trade commitments
4) Develop a two-stage balancing operator for settling unmatched orders in the local P2P market after clearing and handling the deviation between actual operation results and forecasted results. This deviation is first matched internally and then covered by the main grid.
5) Provide a supporting database, dashboard and deployment framework to integrate all sectors as a complete end-to-end system, including real-time results monitoring and scalable execution across

multiple laptops.

The project is not limited to modelling energy consumption alone, instead this project intends to represent a smart energy framework which forecasts, generates energy locally, trade and minimises grid reliance and delivers financial benefits to households.

2 System architecture (JE, GH, JL, YD, GY)

The emulation framework is organised into a modular system in which each function is represented as a separate but connected module. At a high level, the framework includes Household agents, a Market Operator, a Balancing Operator, an orchestrator, a database and a dashboard. These modules represent the core processes required for household energy management, local market participation, balancing, settlement, recording and performance monitoring. **Fig. 1** shows the system architecture in the emulation, and interactions between main modules in the simulation. In practical deployment, these functions could represent the different stakeholders or operational roles.

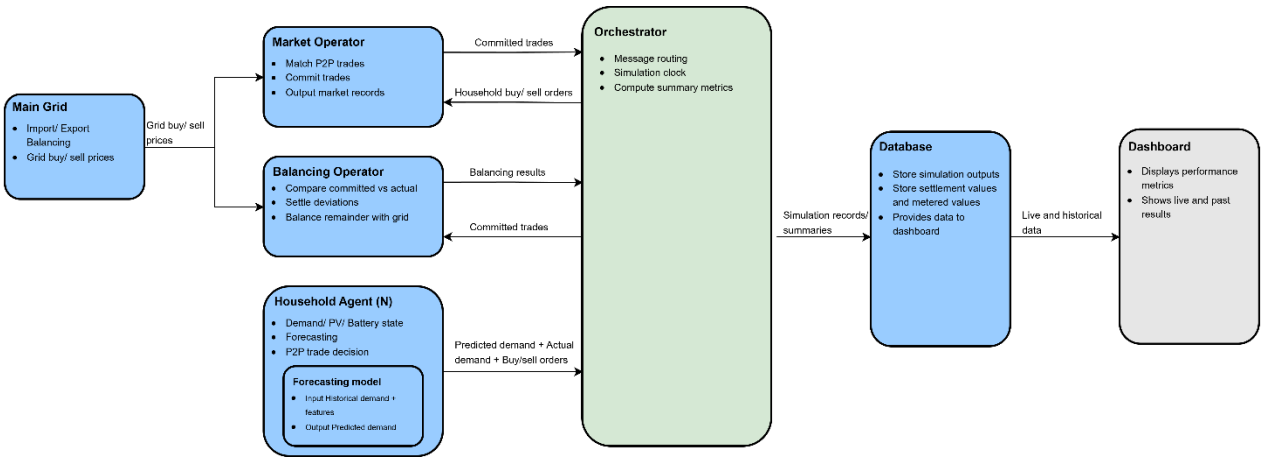


Fig. 1 System architecture for emulation

The core of the system is the Household Agent, representing the household demand, local PV generation, battery operation and P2P market participation logic. Within this module, demand forecasting is used for decision making support for P2P trading, as it estimates likely short-term energy surplus or deficit positions.

The market operator links household forecasting with the balancing and settlement within the system. It coordinates the local P2P trading by receiving the household energy surplus or deficit, forming orders and clearing them using the selected market mechanism. It outputs the committed trades and the unmatched orders.

The balancing operator is responsible for handling all imbalance issues in the entire system, so that all the electricity that households need to solve can be resolved.

The orchestrator acts as the coordinator of the system, coordinating simulation time, collecting household outputs, passing data through the market and balancing stages, and then sending processed results to the database.

The database stores the simulation data in a structured form, allowing system outputs to be preserved and accessed.

The dashboard provides a real-time visualisation for simulation progress, energy flows, costs and household behaviour, enabling the overall system performance to be observable

3 Forecasting model (JE)

3.1 Motivation

In the proposed P2P microgrid emulation, households participate in a local energy market by buying and selling energy according to their forecasted short-term net energy position. To do this proactively rather than reactively, a household must estimate its future demand, compare this with expected PV generation and determine whether it is likely to face a short-term surplus or deficit. Forecasting therefore acts as a functional input to market participation. More reliable forecasting can support better buy or sell decisions and P2P trading can increase local energy sharing and reduce reliance on grid imports [2] [3]. **Fig. 2** shows the role of forecasting within the household P2P decision process.

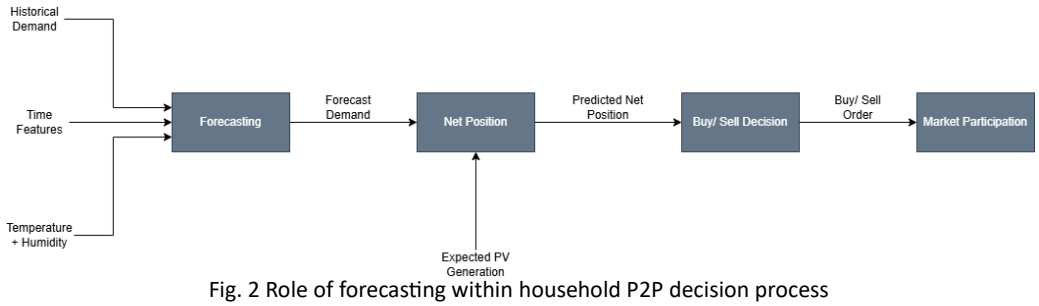


Fig. 2 Role of forecasting within household P2P decision process

3.1.2 Challenges of Household Load Forecasting

Short term household load forecasting is challenging as individual household demand is highly variable and strongly influenced by occupant behaviour and appliance switching [4]. Unlike aggregated demand, household-level demand does not benefit from smoothing across many users, so rapid fluctuations remain and are harder to predict [4] [5]**.** Additionally, smart meter data is privacy sensitive [6], making fully centralised cross-household training unsuitable for realistic deployment, even if it offers strong performance [7]**.**

The aim of this section was to identify a suitable lightweight and deployable short-term household forecasting pipeline for integration into the microgrid emulation. Several neural network architectures were compared across centralised, localised, federated and fine-tuned training methods. Clustered federated learning, increased communication rounds and a reduced training data test were explored as extension studies. The complete python code for implementation of models and experiments can be found at [8]

3.2 Problem formulation, Dataset and Preprocessing

3.2.1 Forecasting Task Definition

The forecasting task is defined as a multistep time-series forecasting problem shown in (1). For each household, the model receives the previous 24 hours of demand and contextual input features, outputting the predicted demand for the next 6 hours in 1-hour intervals. The input features consist of household energy demand, temperature, relative humidity, time encodings and a weekend flag.

$$X_t = \{x_{t-23}, x_{t-22}, \dots, x_t\} \rightarrow Y_t = \{y_{t+1}, y_{t+2}, \dots, y_{t+6}\} \qquad \textbf{(1)}$$

Where X_t is the input window, x_{t-n} is the feature vector at timestep t-n and Y_t is the 6-step output window.

A 6-hour horizon was chosen as provides enough time for households to make proactive trading decisions, a shorter horizon would be easier to predict, but less useful for market participation.

3.2.2. Dataset and Study Period

Energy demand data was taken from the Low Carbon London dataset, containing half-hourly smart meter readings from 5,567 London households between November 2011 and February 2014 [8]. Weather data from the same period was taken from the CEDA archive using records from the London Heathrow weather station [9]**.** Heathrow was used as prior work reported fewer missing weather records [10]**.**

The study period covered **01-01-2012** to **27-02-2014**, giving approximately 800 days of data. This length was chosen to reserve a full 365-day unseen test set for the final emulation while leaving enough data for training and validation sets. This choice is aligned with prior work showing limited improvement from extended training data beyond one year [4].

Households on variable time of use (ToU) Tariffs were filtered out, leaving only standard fixed-tariff households. This was done as variable tariffs may lead to price driven behaviour, which could distort the forecasting task. Invalid timestamps, duplicates and missing values were removed before aligning households to a consistent half-hour structure. The final 100-house cohort was sampled through rule-based filtering for unlikely or inactive demand profiles inspired by [11], followed by manual quality checking after random sampling. The full filtering and replacement details are provided in the eLogbook.

3.2.3 Split Design, Aggregation and Feature Engineering

The dataset was split into training, validation and test sets chronologically. The final 365 days were reserved for the unseen test set for the emulation simulation. The remaining data was split into **80/20** training and validation sets. All architecture and regime comparisons in this section were based on the validation set to prevent biased model selection.

After cleaning the data, missing values were forward filled where necessary and aggregated to hourly resolution, reducing the computational cost for training. The chosen feature set included household demand, temperature, relative humidity, cyclical time encodings, and a weekend flag as prior work showed that time encodings and weather conditions play an important role in household load forecasting performance [4] [10]**.**

Time of day, day of week, and time of year were encoded using sine and cosine transformations so that the model could learn cyclical patterns more effectively using (2), (3), (4) respectively.

$$\boldsymbol{hour_{sin}} = \mathbf{sin}(2\boldsymbol{\pi}\frac{\boldsymbol{s_d}}{\mathbf{86400}}) \ , \ \boldsymbol{hour_{cos}} = \mathbf{cos}(2\boldsymbol{\pi}\frac{\boldsymbol{s_d}}{\mathbf{86400}}) \qquad \textbf{(2)}$$

$$\boldsymbol{dow_{sin}} = \mathbf{sin}(2\boldsymbol{\pi}\frac{\boldsymbol{d_w}}{\mathbf{7}}) \ , \ \boldsymbol{dow_{cos}} = \mathbf{cos}(2\boldsymbol{\pi}\frac{\boldsymbol{d_w}}{\mathbf{7}}) \qquad \textbf{(3)}$$

$$\boldsymbol{year_{sin}} = \mathbf{sin}(2\boldsymbol{\pi}\frac{\boldsymbol{s_y}}{\mathbf{365\times86400}}) \ , \ \boldsymbol{year_{cos}} = \mathbf{cos}(2\boldsymbol{\pi}\frac{\boldsymbol{s_y}}{\mathbf{365\times86400}}) \qquad \textbf{(4)}$$

Where s_d is elapsed seconds within the day, d_w is the day of week and s_y is elapsed seconds within the year.

As Neural Networks are sensitive to the scale of input variables. Household demand was scaled using min-max scaling (5) applied per household using training-set values only, preventing data leakage.

$$x_{scaled} = \frac{x_i - x_{\min}}{x_{\max} - x_{min}} \qquad (5)$$

This improved training stability and allowed the models focus on demand shape rather than absolute magnitude differences across households. Weather variables were scaled using global training-set values, since they were common and public.

3.3. Experimental Design and Evaluation Methodology

3.3.1 Training Regimes

Five training regimes were compared: centralised, centralised + fine-tuning, localised, federated learning (FL) and federated learning plus fine-tuning. Centralised training provided an upper-bound benchmark where household data can be pooled. Localised training represented the fully private case where each household trained independently. Federated learning and its fine-tuned variant provided privacy aware collaborative approach where household collaboration could be achieved while keeping raw data private [10].

3.3.2 Federated Learning Setup

In the federated setting, a shared global model was sent to selected households, trained locally on each household's data, and then returned to a central server where model weight updates were aggregated. Repeating this process over multiple communication rounds allowed collaborative training without direct sharing of raw smart-meter data [10] [7]**.**

The FedAvg algorithm was used because it has relatively low communication overhead compared to other algorithms such as Fed-SGD [7]. A fit fraction of 0.3 was used, meaning that 30% of households were selected in each communication round, and each selected household trained locally for one epoch before aggregation. One local epoch was chosen to reduce the risk of client drift, where the local model overfits to household-specific data and moves too far away from the shared objective.

It should be noted that the FL implementation in this study preserves privacy with respect to raw data, but did not include stronger privacy mechanisms such as differential privacy or secure aggregation as discussed in [7]. As a result, information can still, in principle be inferred from model updates [12] [13], although this is outside the scope of the present study.

3.3.3 Candidate Models and Training Settings

The candidate model families were selected to be lightweight, diverse and consistent with prior household forecasting literature [7]. They included Dense/MLP, LSTM-based, hybrid CNN-LSTM and encoder-decoder style models. Full architecture details are provided in the eLogbook.

The main training settings are summarised in **Table 1**.

	Main model training	Model fine-tuning
Batch size	256	256
Learning rate	1e-3	1e-4
Maximum epochs	500	100
Early stopping patience	20	10

Table 1 Training settings

Adam optimiser was used for both main training and fine-tuning

3.3.4 Evaluation Metrics

Performance was primarily reported using mean RMSE (6) and mean MAE (7) in raw kWh, in line with the reporting style commonly used in the literature. Median values across households were also computed as an indicator to determine whether the mean was affected by a small number of outlier households, and these results are provided in the eLogbook. Raw kWh was used for performance comparisons as the final emulation calculations are based on raw kWh values.

$$RMSE = \sqrt{\left(\frac{1}{n}\right)\sum_{i=1}^{n}(y_i - x_i)^2} \qquad \textbf{(6)}$$

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - x_i| \qquad \textbf{(7)}$$

Errors were computed at each forecasting timestep $t+1$ to $t+6$, averaged per household and aggregated across the cohort. To ensure fairness, all architecture and training-regime comparisons were carried out using the using the same cohort of households, feature pipeline and split design. The test set was not used during the architecture and training regime comparison stage.

3.4. Core Results and Model Selection

3.4.1 Centralised Architecture Screening

Centralised screening was used first as it represents the ideal case where privacy is not a constraint and all household data can be pooled. This provides a useful benchmark and allows weaker architectures to be removed before more computationally expensive FL experiments.

Under centralised training, the leading architectures performed within a narrow RMSE range (0.3438-0.3463), showing limited difference based on predictive performance. As a result, the shortlist was chosen based not only on RMSE, but also on architectural diversity and computational practicality. The four models carried forward were **CNN-LSTM, CNN-LSTM-Dense, LSTM64_32 and LSTM64_Dense**. The dense-only baseline performed noticeably worse, supporting the importance of time sequence modelling for this task.

3.4.2 Localised and Federated Baselines

For localised training, in addition to the shortlisted neural models, a persistence baseline and a SARIMA (Seasonal Autoregressive Integral Moving Average) model were included. Persistence provided a naïve reference, while SARIMA provided a lightweight traditional forecasting benchmark, although often argued to be less effective for capturing non-linear patterns and non-aggregated data [7]**.**

Federated learning was then evaluated using the FedAvg setup described in Section 3.3, with a fit fraction of

0.3 and 200 communication rounds.

The Localised and FL-only baseline results are shown in **Table 2**.

Model	Localised RMSE	Localised MAE	FL RMSE	FL MAE
SARIMA	0.3309	0.2093	NA	NA
CNN-LSTM	0.3393	0.2198	0.3892	0.2708
LSTM64_32	0.3399	0.2202	0.3832	0.2628
CNN-LSTM-Dense	0.3406	0.2188	0.3860	0.2674
LSTM64_Dense	0.3412	0.2203	0.3911	0.2729
Persistence	0.5213	0.3426	NA	NA

Table 2 Localised and Federated baseline results

Persistence performed considerably worse than all the other models, confirming that predictive modelling was justified. Among the localised models, SARIMA was the strongest benchmark at this stage, showing that household-specific statistical modelling remained competitive even without information sharing. Conversely, FL-only training underperformed both centralised and localised baselines, likely reflecting the non-IID nature of household demand and the limited 200-round budget.

Fig. 3 shows the validation RMSE trend over communication rounds for the shortlisted models in the FL-only setting. To manage computational cost, validation RMSE was computed in 5-round chunks.

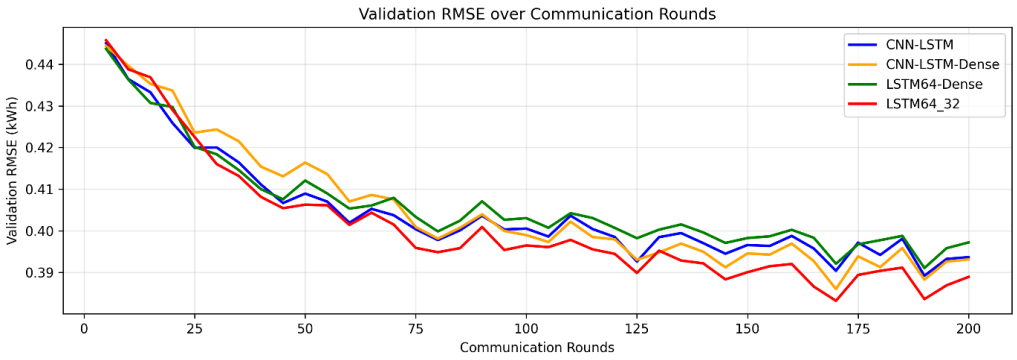


Fig. 3 FL validation loss

The validation curves for all models continue to decline toward the end of the training run, suggesting the original 200-round budget was insufficient for optimal performance.

3.4.3 Effect of Local Fine-Tuning under Centralised and Federated Pretraining

Local fine-tuning was then evaluated under both centralised and federated pretraining to determine whether a shared pretrained model could be adapted to individual household behaviour, with results shown in **Table 4**.

Model	Centralised + FT RMSE	Centralised + FT MAE	Federated + FT RMSE	Federated + FT MAE
CNN-LSTM	0.3255	0.2034	0.3308	0.2128
CNN-LSTM-Dense	0.3261	0.2027	0.3305	0.2107
LSTM64_Dense	0.3260	0.2037	0.3298	0.2110
LSTM64_32	0.3266	0.2040	0.3276	0.2091

Table 4 Fine-tuning results

To visualise these fine-tuned results in the context of the full training-regime comparison, **Fig.4** shows the mean RMSE of the four shortlisted models across the five training regimes.



Fig. 4 Training regime

Fig.4 shows that although small, the training regime had a stronger effect on performance than the choice of shortlisted architecture. Across all four models, federated learning alone gave the highest errors, while fine-tuning substantially improved both the centralised and federated pipelines. In particular, federated fine-tuning recovered much of the performance lost under FL-only training and brought the shortlisted models to within a narrow margin of the centrally pretrained fine-tuned results while retaining its privacy advantage. These results suggest that cross household training can help the model learn broader household patterns, while local fine tuning can adapt the model to the specific household behaviours.

Within the shortlisted models, **LSTM64_32** achieved the strongest overall federated fine-tuning performance. Fine-tuning reduced the median household RMSE from 0.324kWh to 0.293kWh, lowered the lower quartile and reduced the number of outliers, indicating that fine-tuning helped the worst performing households increase predictive accuracy.

3.4.4 Final Model Selection

The final forecasting pipeline was selected based on predictive accuracy, privacy suitability and computational practicality with household-level deployment. The chosen model was therefore **LSTM64_32** trained using federated learning with local fine-tuning. This pipeline offered the strongest balance between deployment feasibility and predictive accuracy. Although centralised + fine-tuning achieved slightly stronger overall results, it required household data pooling and was therefore not considered suitable for practical deployment. Localised SARIMA remained competitive and lightweight, but it did not exploit cross-household learning.

3.5. Extension Studies

3.5.1 Clustered Federated Learning

Clustered FL was investigated as a possible way to reduce the effect of non-IID household behaviour [10]. Households were represented using averaged 24-hour daily demand profiles from the training split following [14] [15], then unit normalised using **(8)** so that clustering focused on profile shape rather than magnitude.

$$y_{scaled} = \frac{y_i}{|y_i|} \qquad (8)$$

Where y_{scaled} is the scaled feature vector and y_i is the original feature vector.

K-means clustering was used because it is simple, and commonly used in residential load-profile studies [14] [15]. A sweep from K = 2 to K = 6 was evaluated using inertia, silhouette score, Davis-Bouldin index, Calinksi-Harabasz score and cluster sizes. Larger values of K were not explored to control compute cost and because the expected additional benefit was limited [14].

For simplicity, clustering was performed directly on household load profiles rather than on privacy-preserving representations such as gradients updates as discussed in [10].

The clustering results are shown in **Table 5**

K	inertia	Silhouette	DBI	CH	Min cluster size	Max cluster size
2	7.400	0.2364	1.6287	22.7	24	76
3	6.243	0.1724	1.6570	22.3	20	46
4	5.264	0.1776	1.4315	23.4	2	43
5	4.810	0.1637	1.5299	21.3	2	28
6	4.429	0.1648	1.3363	19.9	1	28

Table 5 Clustering results

To interpret the results beyond numerical metrics, **Fig. 5** shows the centroid profiles for **K =2, 3, 4**

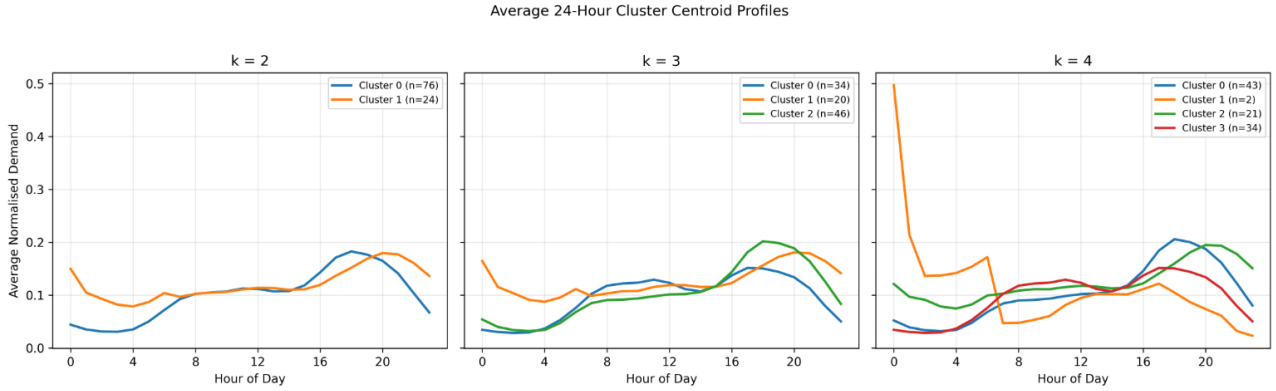


Fig. 5 Cluster average profile plots

The centroid profiles differ in shape, notably in the timing of daytime and evening peaks, although all groups kept a similar pattern with lower late-night usage and higher daytime and evening usage, suggesting that while meaningful behavioural differences exist, most households follow a common daily rhythm. For K = 4, one cluster contains only 2 households, indicating that a small outlier group of households was captured rather than a distinct group.

The clustered FL and FL plus fine-tuning results are shown in **Table 6**, LSTM64_32 was used for this experiment.

K	FL RMSE	FL MAE	FL + FT RMSE	FL + FT MAE
2	0.3754	0.2548	0.3274	0.2089
3	0.3796	0.2549	0.3282	0.2093
4	0.3717	0.2487	0.3294	0.2093

Table 6 Clustered FL results

Clustered FL-only results showed a small improvement over unclustered FL. However, once local fine-tuning was added, the advantage became negligible and disappeared in some cases. This suggests that clustering may improve the shared model, but local fine-tuning addresses much of the household-level variation. Clustered FL was therefore treated as an exploratory extension rather than the main deployment choice.

3.5.2 Increased Communication Rounds

Since the initial FL validation curves still showed a downward trend at the end of the original 200-round budget, the best performing unclustered FL pipeline, LSTM64_32, training was extended to 500 communication rounds, with results shown in **Table 7**.

Training regime	RMSE	MAE
FL only	0.3708	0.2429
FL + FT	0.3243	0.2053

Table 7 Increased communication rounds RMSE

Increasing the communication budget improved both the FL-only model and the FL plus fine-tuning pipeline, strengthening the chosen unclustered FL + fine-tuning approach without introducing extra complexity into the system. However, longer FL training also increases runtime cost and model communication between households is a practical bottleneck in real life FL settings [10] and is desirable to keep this as low as possible.

3.5.3 Limited Data Test

A separate limited-data test was carried out to assess how well the FL-based approach adapted to new households with limited local history available. This was especially relevant as SARIMA remained competitive in the localised baseline and is substantially lighter to deploy.

For this experiment, a separate set of 10 households was sampled from the eligible pool, excluding both the main 100 house cohort and the previously rejected households. For each household, a fixed 2-week window was extracted and split into 5 days of training, 2 days of validation, and 7 days of test data. The methods compared were SARIMA, the pretrained FL global model and the pretrained global model with local fine-tuning, the results are shown in **Table 8**

Training regime	Mean RMSE	Mean MAE
Pretrained FL	0.4530	0.2874
Pretrained FL + FT	0.4352	0.2826
SARIMA	0.4853	0.3047

Table 8 Limited data training results

Both FL-based methods outperformed SARIMA, supporting the final selected pipeline by showing that collaborative pretraining is beneficial when only a short local history is available for adaptation.

3.6 Conclusion

This section has shown that short term load forecasting is a necessary input for market participation within the P2P microgrid emulation. While centralised training provided a useful upper-bound benchmark, it is not realistic for deployment under practical privacy constraints. Localised and federated approaches therefore

provided more realistic deployment pathways.

Among these, federated learning with local fine-tuning offered the strongest overall balance between predictive accuracy, privacy suitability, and deployment practicality. Within that regime, **LSTM64_32** was the strongest architecture. Increased communication rounds further improved its performance. Clustered FL provided a useful extension study but did not overturn the main deployment choice. Finally, limited-data test further strengthened the selected pipeline by showing that FL-based pretraining remains useful when adapting to newly encountered households.

4 Household Agents (GH)

As stated before, households in an energy distribution system will be a prosumer rather than a simple energy consumer. For this to happen, a software agent is required to be capable of modelling household behaviour in a realistic manner.

In the project architecture, the household agent is one of several interacting components that requires to behave as close as possible to real life households that will act as active participants like mentioned before. With a working and realistic model working, the household agent is also required to communicate planned energy quantities to the Market operator, report actual metered values and use predicted data from machine learning to forecast.

4.1 Planning and design process

4.1.1 Role of the Household agent

The planning process began by first defining the role of the household agent in the project architecture. The household agent has to act active participants rather than a simple passive load in the simulated system. This meant that the agent must capture the electricity consumption, local energy generation, storage, grid interaction and control strategy [16]

The key design objectives will be to create a household model that is realistic enough to reflect household-level behaviour, while remaining practical for implementation for the overall project. Not only does the model need to represent realistic physical of how a household energy system would behave, the way of energy flows and decision-making processes need to be physically meaningful and logically consistent for the project.

4.1.2 Household energy generation

This model will mainly focus on PV generation as the energy generation for households. For household-level energy studies, rooftop PV is a very common choice due to its already wildly used in homes. The IEA notes large existing households are using PV as a source of energy generation, which makes PV a realistic basis for a household agent model [17]

Wind energy was also considered as a possible renewable source. However, wind generation is more commonly represented at community, farm, or grid level rather than at the level of an individual household. Modelling wind at household scale would require additional assumptions and add more complexity into the

project. The design approach to this project also wants to treat each household as an independent agent with its own local demand, energy generation and battery behaviour, PV only generation was selected as the more suitable option.

4.1.3 Functional decomposition

The household agent is divided into smaller functional components so it's easier to plan, design and test. The decomposition is necessary because realistic household energy management also depends on the interaction between several different processes rather than one single calculation [16] The model will be separated into demand, generation, battery storage, control, forecasting and output functions. This helps the developing process in general where each part can have its own clear purpose and easy to debug later.

Table 9 summarizes the main functional components of the household agent and the objective each component wants to achieve in this model.

Components	What it needs to achieve
Household demand	Must provide realistic household demand in real KWH values. Need to be compatible with forecast integration
PV generation	Must represent realistic household level generation from real life irradiance data. The irradiance data need to be converted into actual PV output Must remain consistent with a fixed PV setup per household during each run
Battery storage	Must represent battery charging, discharging and state of charge. Must have capacity, charging or discharging limits, efficiency and state of charge bounds.
Household agent	Must decide when to discharge and charge based on demand and battery storage. Later it must integrate with forecasting component.
Forecast input	Use prediction value to adjust battery behaviour. Estimate future PV availability for forecasting. Calculate future net need.
Output functions	Must calculate and output household import and export energy into the grid. Have clear visuals of demand, PV, battery, import and export.

Table 9

4.1.4 Early design approaches

Real-life model

Different implementation routes were considered for modelling household. Building an actual real-life small PV system was considered because it's the easiest way to achieve a realistic model. However, this project would require a huge amount of household. Each individual household would have different parameters and will add a lot more complexity not only to developing the model, also merging with other parts in the project. Therefore, building a real-life model is scratched.

Simulink

Another consideration would be to use Simulink on MATLAB to design the PV, then output the values to the household where calculations will be carried out to find total energy remaining. Simulink can provide a more detailed and more realistic PV model due to its massive library of components. A realistic household

management system with rooftop PV will have inverters because electricity generated from solar panels are DC however household and grid will have to operate on AC electricity. In this case Simulink is a good option to model a PV system with such requirement. With inverters and other necessary components included in the system, they will therefore influence the efficiency of the system. With these factors, the output result will be more realistic and therefore more accurate results for the whole project.

Compared to the real-life model, Simulink will be easier to build, debug and implement. However, it has similar issues due to the project requires a large number of household and different size household would require different parameters of PV (small household can't have a massive PV or it will be unrealistic.). Simulink can provide subsystems to have different household with corresponding demand but from the scale of this project which we aim to test at least 20 households, it will still be time consuming to build different subsystems that read different data and not efficient when trying to find which part went wrong.

Python

Python is the most suitable implementation environment for this model after many considerations. The household agent model needs to fit into a software-based architecture rather than just a standalone model. Python is a good choice because it is easy to use across multiple modules and can integrate with other parts of this project.

Python also gives more freedom to build the model as the objectives this model wants to achieve. The household agent is divided into different components and can be easily implemented in python. There will be some limitations for how realistic python can make this model, but it can perfectly satisfy this project's requirement.

For this project we have decided to read real hourly London household demand data and hourly irradiance data between 2012 – 2014. Python can read and merge big scale of data with pandas. And therefore, it can be easier to test different households with different parameters.

4.1.5 Model overview

With every component introduced, the general structure of the household agent model is shown in **Fig 6:**

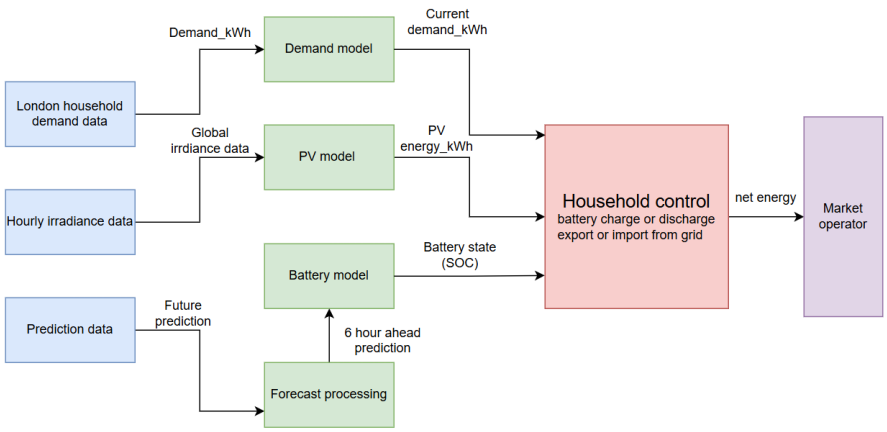


Fig. 6: Household agent structure

4.2 Battery modelling

4.2.1 Battery purpose

Battery storage is an important component of the household energy management system because it aids generated energy to be shifted over time and be used at suitable time based on demand [18]. The simple logic would be the battery allows surplus electricity to be stored and use stored electricity based on deficits or demand of the household.

This improves the flexibility of the household energy system; it reduces the reliance on constant grid transport and increases the use of local energy generated from PV. Additionally, the battery can be further improved by forecasting with better decision making.

4.2.2 Parameters and structure

Selecting parameters

The main battery parameters are based on physical properties required to represent realistic battery behaviour for a household energy management system and are shown in **table 10**. These parameters determine how much energy the battery can store, how quickly it can charge and discharge and the battery limits.

Parameters	Meaning
Capacity (kWh)	Total battery storage capacity
SOC	Current state of charge of Capacity (%)
Max charge (kw)	Maximum charging power of the battery
Max discharge (kw)	Maximum discharging power of the battery
Efficiency	Charge and discharge efficiency
SOC max	Maximum allowed SOC
SOC min	Minimum allowed SOC

Table 10: Parameters for battery

Suitable parameters

Because this project would require testing on a large amount of household with different parameters, it was not appropriate to assign the same battery parameter to every household. In practice, a small household with low demand would be unlikely to have a battery system clearly for large households; therefore, the battery model needs to adapt according to the size of the household. This model will assume each household will have directly proportional battery size to the household size. To find out the size of the household, the assumption was made through the average yearly demand of the household, and each household was placed into a demand band from which battery capacity were assigned. A limitation of this approach is that average yearly demand does not fully capture the household consumption behaviour. Two households with similar average demand can still have different daily or monthly consumption which can justify different battery sizes. However, this method can be considered as a reasonable simulation rule rather than optimised real-world design approach due to how time consuming it can be by just selecting the data.

Information about household size matching battery size is found on [19].

4.2.3 SOC and stored energy

The state of charge (SOC) of this battery model is defined as a fractional value between 0 and 1, where 0 represents empty and 1 represents fully charged. However, in practical, battery is not allowed to reach extreme level [20]So, in this model the SOC will set a limit between 0.2 to 0.95.

To convert SOC to physical energy values, an equation to calculate current stored energy based on SOC and maximum energy capacity can be found:

$$E_{stored} = SOC_{max} \times C \quad \textbf{(9)}$$

Where instead of maximum energy capacity, the model will use maximum SOC.

4.2.4 Charging logic

The charging logic takes place when there has been a surplus energy from the household, it will determine how much surplus energy can be absorbed by the battery during a simulation step.

Limited charging power

The requested charging energy will be limited to a limited range between zero and the maximum charging power of the battery. This can prevent unrealistic charging behaviour and make sure the battery doesn't absorb energy faster than its maximum charging power.

Since the model is tested in time-step, the charging power would require transferring into energy which can use the equation:

$$E_{in} = P_{charge} \times t \quad \textbf{(10)}$$

t represents the simulation time step and E_{in} is the input energy into the battery.

Available room in battery

The model later will calculate the remaining storage room in the battery. This is done by comparing the current stored energy to the maximum allowed stored energy which is defined by the upper SOC bound.

$$E_{room} = E_{\max allowed} - E_{stored} \quad \textbf{(11)}$$

If there is no available room left in the battery, the charging function returns to zero and no further charging will take place.

Efficiency within battery

As discussed before, in real life home energy system there will be losses due to inverters or other factors. Same with battery, part of the incoming energy is lost during the charging process. This model will assume each battery will have an efficiency of 95% because usually modern household batteries efficiency goes around 90% to 97% [21].

With efficiency included, only a part of the input energy will be stored, there will be an actual stored energy calculated:

$$E_{actual,stored} = E_{in} \times \eta \quad \textbf{(12)}$$

This makes charging more realistic since it can show energy storage is not lossless.

Maximum allowed charging input

Because charging losses are considered, even after efficiency loss, there is a chance the input energy will exceed the available room in battery. Therefore, the maximum allowable charging input is determined by:

$$E_{input,max} = \frac{E_{room}}{\eta} \quad \textbf{(13)}$$

Charging logic summary

Once the actual stored energy is calculated, the SOC will be updated to reflect the energy level at last. The overall charging logic can be summarized as:

- Take requested charging power
- Make sure the charging power doesn't exceed over maximum battery storage or below zero
- Calculate how much energy from this time step
- Reduce it by efficiency
- Check room left on battery
- Store based on how much energy fits

4.2.5 Discharging logic

The discharging logic uses the same principle and equations as the charging logic but in reverse. Instead of storing surplus energy, it determines how much useful energy can be delivered to household base on demand. It can be summarized as:

- Take requested discharging power
- Make sure the discharging power does not exceed maximum discharging power or fall below zero
- Calculate how much energy is needed in time step
- Check how much useable energy is available
- Apply discharge efficiency
- Deliver only the amount of energy that the battery can supply within its limits

4.3 PV generation model

4.3.1 PV selection based on household

PV will have the same issue as the battery with the parameter values selected for each household. So, the PV will take the same approach where the area of the PV will adapt to the size of the household, and they will be also chosen based on the average demand band.

4.3.2 Calculations

The data used for PV is hourly irradiance in London with the same dates as the household demand data. The hourly irradiance will be converted to solar energy by:

$$solar\ energy = \frac{irradiance}{3600} \quad \textbf{(14)}$$

This conversion was used because the irradiance data was provided as hourly irradiation in kJ/m^2, divided

by 3600 coverts the value into kWh/m^2 for further calculations.

The solar energy calculated will be used to calculate the energy generated by PV:

$$PV_{energy} = solar\ energy \times\ pv\ area \times\ efficiency \times\ performance\ ratio \quad \textbf{(15)}$$

The efficiency represents the panel conversion where in [22] stated the average panel conversion are between 15% to 24%, so in this project will use 21% as a good middle ground for testing. Same research also stated the performance ratio for real PV system usually sits around 80% to 90%, therefore the model will use 85% as performance ratio.

4.4 Agent model

The household agent acts as the main decision-making unit for each household in this model. It takes all the results from other components and determines if the household should charge or discharge. If there is further surplus or deficit in energy, the agent needs to decide whether to export or import from grid.

4.4.1 Calculations and decision making

To determine whether to charge or discharge battery, the net energy is calculated through:

$$Energy_{before} = demand - PV_{energy} \quad \textbf{(16)}$$

Where the demand is loaded from the demand model. A positive net energy indicates that household demand is greater than local generation, therefore the household would require to supply from battery (discharge battery). A negative value indicates the PV energy exceeded demand, creating surplus energy that can be stored (charge battery).

After the battery action, new net energy needs to be calculated so the agent can decide whether the battery has enough to supply or the surplus exceeded the battery storage.

$$Energy_{after} = demand - PV_{energy} - battery_{discharged} + battery_{charged} \quad \textbf{(17)}$$

If the net energy calculated from this equation is bigger than zero, then import to grid. If it is smaller than zero, then export from grid. The net energy will be sent to the market operator for further action.

4.5 Forecast processing

4.5.1 Purpose of forecasting

Forecasting is introduced for the household to make smarter decisions to achieve the purpose of this project which is a smart micro-grid. The model without forecasting will only react to current household's conditions whereas a forecast model can help the household to consider expected future energy demands.

4.5.2 Logic with forecasting

With forecasting, the model will look at future predictions and decide based on if the deficit will be too high or surplus is high. If future deficits are high, the battery will keep more energy now and avoid discharging too much. If future deficits are low or surplus are high, the battery will be discharged more freely, or net energy will be exported now.

4.5.3 Predications

Demand predictions

The future predictions values for household demand are given by the machine learning component. The values are given over a 6-hour prediction horizon, where each output represents the expected household demand for a future time step. To simplify the use of forecast information, the 6-hour prediction is summed into one singular value (Eq.18).

$$future_{demand} = pred_{1h} + pred_{2h} + pred_{3h} \ldots. \quad \textbf{(18)}$$

PV predictions

In addition to future household demand, a future PV prediction is also required. A separate PV forecasting model was not built for this project because it will be time-consuming and will be more like a side project other than our main objectives. For this reason, the PV will also have a summed value for future 6-hours prediction and the values will be used directly from the future PV generation series.

4.5.4 Priority of exporting

In the battery control logic, surplus energy is not automatically prioritized for battery storage, instead the model gives priority to trading or exporting surplus energy before charging the battery. This design choice was due to early testing; some seasons will have unbalanced PV generation compared to demand. In such periods, always storing store first caused the battery to fill quickly and reduced amount of energy for P2P trading. By prioritizing trading first, the model allows generated energy to contribute more actively into the market.

Although trading is prioritized, it doesn't mean local batteries won't store surplus energy. Instead, the charging is controlled to take place when local storage is considered necessary based on future demand from forecasting model. It will look at forecasting data and only charge to a defined level rather than the first approach where it will always be fully charged.

4.5.5 Results from forecasting

Fig 7 shows the SOC of the battery without forecasting and prioritizing exporting and **Fig 8** shows with forecasting and prioritizing exporting for household MAC000018:

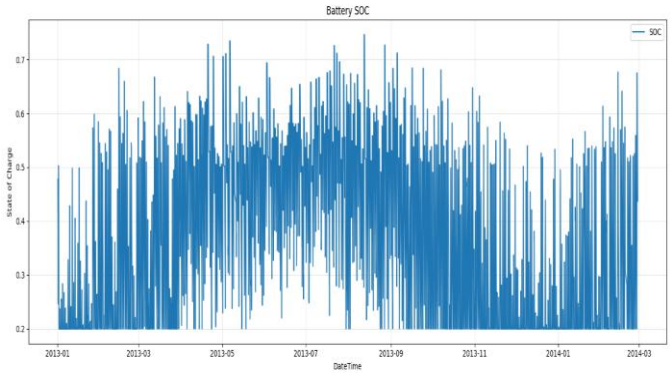


Fig. 7

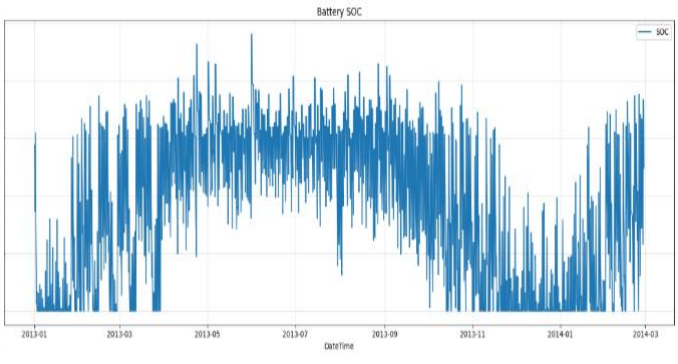


Fig. 8

It shows clearly the SOC is charging and discharging more conservatively and consistently.

The household agent model can be accessed on [23] and result from household (demand, PV and net export) can be found on my section in the E-logbook.

5 Market Operator (GY)

This chapter discusses the role and design of the market operator. It receives the forecasted import/export energy from households, organizes local P2P trades and then generates market commitments. In this system, it links the forecasting with the subsequent balancing and settlement.

5.1 Market design and related work

P2P power trading is classified into full P2P, community-based P2P, and hybrid P2P [2]. In this project, the market operator needs to match the forecasted household energy within the community and generate the trade commitments for subsequent use by the balancing and settlement modules. So full P2P cannot be adopted because although it provides high trade autonomy, it has coordination problems. The hybrid P2P structure can use the indirect coordinator and preset signals, but has limited adaptability to real-time network conditions. Therefore, community-based P2P is used because it has stronger coordination capabilities by using an intermediate coordinator to organise communication and trading. The market operator acts as the intermediate coordinator to unify market clearing, trade commitments, and connection with subsequent balancing and settlement processes. However, this structure also has some limitations, which are discussed in Section 8.

Also, the main difference between P2P markets and traditional energy markets is the market and pricing mechanism [3]. Pricing mechanisms commonly use two models: the double auction and the analytical [24]. Analytical model is simple to implement because the prices are directly calculated through mathematical rules. But double auction model forms the trade price through market competition, which can more directly reflect the local demand and supply and the household price preferences. Thus, double auction model is selected for this project for the internal matching within the community.

Furthermore, most of the existing literature focuses on market clearing or pricing itself, rather than integrating forecasting, trade commitment, actual outcome and settlement within a single operational pipeline. But this project, market operator acts as the link between forecasting, balancing and settlement.

5.2 Overview

The market operator acts as the intermediate coordinator of the community-based P2P market. It receives the forecasted import/export energy from each household in each trading slot, forms and clears market orders under the selected mechanisms. The output of the market operator is not the realised physical exchange but the trade commitments that are used by the subsequent balancing and settlement module. **Fig .9** shows the system architecture and data flow of the market operator.

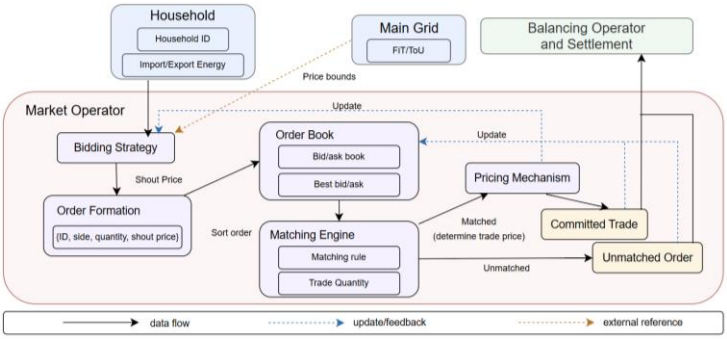


Fig. 9 System architecture of the market layer Fig.

5.3 Order formation and order book

In each trading slot, the market orders are formed based on the forecasted import/export energy. The household with energy deficit forms a buy order, while the household with energy surplus forms a sell order. Each order includes the household ID, side (buy or sell), slot time, shout price and quantity.

Then, these orders are collected into the order book, which is divided into bid orders and ask orders. Bids are sorted in descending order of price, while asks are sorted in ascending order of price. If orders have the same price, they are sorted based on the time priority, following the FIFO rule. **Table 11** gives an example of an order book.

Bid Order				**Ask Order**			
h_id	sub_seq	price (£/kWh)	quantity (kWh)	h_id	sub_seq	price (£/kWh)	quantity (kWh)
H2	4	0.0239	5.8	H3	3	0.0167	4.5
H5	5	0.0239	6.3	H6	2	0.0173	9.7
H1	1	0.0207	8.5	H4	6	0.0198	6.1

Table 11 Example of order book

5.4 Market Mechanisms

5.4.1 Overview of mechanisms

After organizing the buy orders and sell orders, the market operator clears the local market using an auction mechanism. Three market mechanisms are considered: continuous double auction (CDA) [25], call auction (CALL) [26], and uniform-price double auction. Although all three mechanisms share the same order book structure, they have significant differences in clearing rule (matching rule), pricing mechanism, and quantity allocation, which are summarized in **Table 12**.

Among these three mechanisms, CDA is described in detail because its continuous clearing structure is more suitable for this project. The relative performance of these three mechanisms is still compared and evaluated in Section 5.7

Mechanism	**Clearing mode**	**Typical price rule**	**Quantity allocation**
CALL	Orders are collected together and cleared at a fixed time point	Call price	Central allocation
Uniform-price	Orders are collected together and cleared at the end of the period	The intersection point of the aggregate supply and demand curves	Central allocation at a uniform price
CDA	Continuous clearing once the trading conditions are met	Individual trade price for each trade	Individual allocation for each trade

Table 12 Summary of market mechanisms

5.4.2 CDA Design

In the CDA design, the market operator incorporates the formed buy and sell orders into the order book and checks whether the matching rule is satisfied in each trading slot. For the successfully matched orders, the trade price and quantity will be determined. Unmatched orders are returned to the order book for reordering and waiting for the next round of matching. **Fig. 10** illustrates how the market operates within a single trading slot.

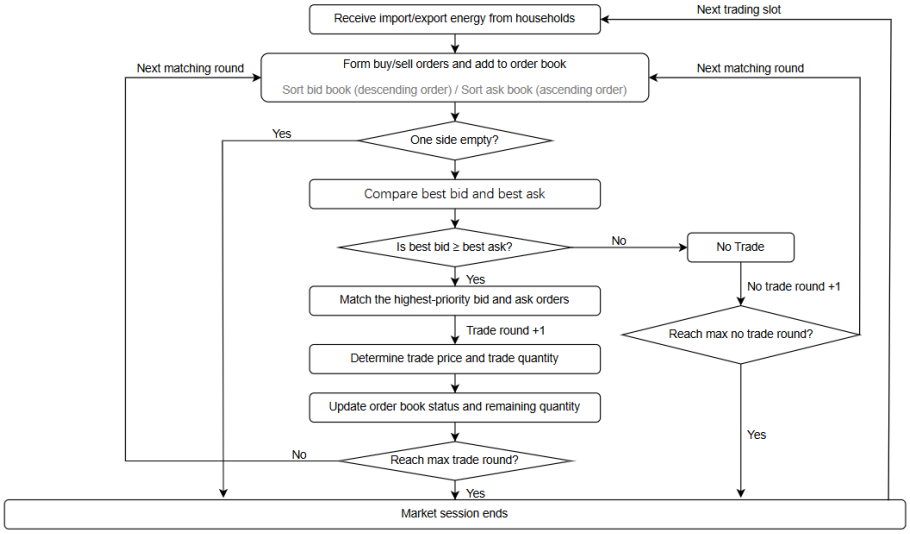


Fig. 10 Flowchart of the CDA market clearing within the single trading slot

5.4.2.1 Matching rule

After the order book is established, the market operator matches the buy and sell orders according to the matching rule. In each round of matching, the best bid at the top of the bid book and the best ask at the top of the ask book are compared. These represent the buyer with the highest willingness to pay and the seller with the lowest acceptable price. The matching rule is as follows:

$$p_{best_bid} \geq p_{best_ask} \tag{19}$$

where p_{best_bid} is the best bid and p_{best_ask} is the best ask.

When a trade is executed, the market updates the order status and remaining quantity, and the order book is reordered for the next round of matching. If the matching rule is not satisfied, then no trade can be generated in this round, and the orders will be carried to the next round for comparison.

When one side of the order book is empty, the market session ends and the remaining unmatched orders are trades with the main grid. The session is also terminated if reaching the market stop conditions: max trade round and max no trade round. This can prevent the matching process from continuing indefinitely.

5.4.2.2 Trade quantity

After two orders are successful matched, the trade quantity is determined by:

$$q_{trade} = \min\left(q_{buy}, q_{sell}\right) \tag{20}$$

where q_{buy} and q_{sell} are the quantities in the buy and sell orders respectively. This is because the trade

quantity cannot exceed either the demand of buyers or the supply of sellers.

Moreover, if the quantities from the buyer and the seller are not the same, the remaining quantity is left for subsequent matching round or be treated as an unmatched order.

5.4.2.3 Pricing Mechanism

Under the CDA mechanism, the trade price can be determined by different rules. **Table 13** summarizes the main pricing rules considered in this project, including the pay-by-ask, pay-by-bid, the midpoint rule and the k-factor rule [27]. They mainly differ in how the profits from trade are distributed between the buyer and the seller. The first three rules can be determined as special cases of the k-factor rule, corresponding to k = 0; k = 0.5; k = 1.

Pricing mechanism	Formula	Favored side
pay by ask	$p_{trade} = p_{ask}$	Buyer
pay by bid	$p_{trade} = p_{bid}$	Seller
midpoint	$p_{trade} = \frac{p_{bid} + p_{ask}}{2}$	Relatively balanced
k-factor	$p_{trade} = kp_{bid} + (1-k)p_{ask} \quad (0 \leq k \leq 1)$	Larger k: seller/Smaller k: buyer

Table 13 Summary of pricing mechanisms

5.5 Bid and ask strategy

After the market operator determines whether the household acts as a buyer or seller, the shout price is generated based on the bidding strategy. Every household wants to maximise their benefits, buyers want to minimize the cost of purchasing energy, and sellers want to maximize the revenue from selling energy. Also, the shout price needs to be constrained by the acceptable price bounds to make sure the market is attractive for both sides. The following sections introduce the three common bidding strategies.

5.5.1 Static Strategy

The static strategy is used as a non-adaptive benchmark bidding strategy, which is the baseline to evaluate whether the adaptive strategy can improve the market performance. Under this strategy, once the buying and selling intentions are determined, the shout price is generated for each household according to the price setting rules. The initial shout price remains fixed throughout the market session and is not continuously updated in response to real-time market feedback.

5.5.2 Zero Intelligence Plus Strategy

The ZIP strategy is a lightweight adaptive bidding strategy designed for the CDA mechanism [26] [28]. Unlike non-adaptive strategies, ZIP updates the shout price based on the recent market feedback. Each household agent sets a limit price and a profit margin, and the shout prices of sellers and buyers are determined by:

$$p_{ask}(t) = l_{ask}\big(1 + \mu(t)\big) \tag{21}$$

$$p_{bid}(t) = l_{bid}\big(1 - \mu(t)\big) \tag{22}$$

where l_{ask} is the limit price for sellers which shows the minimum acceptable selling price and the l_{bid} is the limit price for buyers which shows the maximum willingness to pay. The profit margin $\mu(t)$ determines the gap between the limit price and the final shout price

The ZIP update is based on the market feedback and signals. **Algorithm 1** determines whether to raise, lower or maintain the profit margin according to trade or no trade. The Widrow-Hoff with momentum learning rule is used to update the shout price, and the current shout price is converted to profit margin for the next round:

$$\Delta_{raw}(t) = \beta\big(\tau(t) - p(t)\big) \tag{23}$$

$$p_{new}(t) = p(t) + \Delta(t) = p(t) + \gamma\Delta_{last}(t-1) + (1-\gamma)\Delta_{raw}(t) \tag{24}$$

where $p(t)$ represents the shout price, $\tau(t)$ is the target price, also the reference price formed based on the recent market signal, and β is the learning rate. γ indicates the momentum coefficient. $\Delta_{raw}(t)$ is the raw update amount directly calculated based on the difference between the target price and the current shout price for the current round, while $\Delta_{last}(t-1)$ is the actual price update amount adopted in the previous round.

Algorithm 1: ZIP Profit Margin Update Rule

```
For seller
    If last shout was accepted at the trade price q // trade
        If the ask price p_i ≤ reference price q (trade price) → raise the profit margin
        If the last shout was 'bid' and the ask price p_i > reference price q (trade price) → lower the profit margin
        Else no update
    Else// no trade
        If the last shout was 'ask' and the ask price p_i ≥ reference price q (best ask) → lower the profit margin
        Else no update
For buyer
    If last shout was accepted at the trade price q // trade
        If the bid price p_i ≥ reference price q (trade price) → raise the profit margin
        If the last shout was 'ask' and the bid price p_i < reference price q (trade price) → lower the profit margin
        Else no update
    Else // no trade
        If the last shout was 'bid' and the bid price p_i ≤ reference price q (best bid) → lower the profit margin
        Else no update
```

Algorithm 1

5.5.3 Adaptive Aggressive Strategy

AA is a more advanced adaptive bidding strategy. Compared with ZIP, AA uses both short-term and long-term learning mechanisms to update the shout price based on the market feedback [29] [28]. In this project, a simplified AA strategy that retains the core idea is adopted.

The core state variables of AA include aggressiveness r, the estimated market reference price $\hat{p}$, and the curvature parameter θ. Here, r < 0 indicates a more aggressive approach, r > 0 indicates a more conservative approach, and r = 0 indicates a neutral approach. $\hat{p}$ represents the market reference price estimated based on the recent trade prices; θ is used to adjust the non-linear mapping relationship between aggressiveness and the target shout price. During shout price generation, more aggressive buyers and sellers move the bid and ask closer to their own limit price, thereby increasing the probability of a trade. More conservative behaviour moves the shout price away from the limit price in order to gain higher returns. The shout price is still subject to the price bounds.

In terms of the update mechanism, AA incorporates two levels: short-term learning and long-term learning. Short-term learning updates the trader's aggressiveness based on current market signals; long-term learning updates $\hat{p}$ and θ after a successful trade, thereby updating the estimation of the market state based on the recent trade prices and their volatility.

5.6 Market Outputs

After the market session ends, the output mainly consists of the matched trade commitments and the unmatched remaining orders, which form the basis for the subsequent balancing and settlement.

5.6.1 Committed Trades

Committed trades are the successfully matched orders and formed by market operator through market clearing within each trading slot. They represent the market trading commitments reached by both buyers and sellers within the community and reflect the final matching result of the local market during that trading slot.

5.6.2 Unmatched Orders

Unmatched orders are the buy and sell orders that have not been cleared through internal matching. These include completely unmatched orders as well as the remaining unmatched quantities after partial trades. These situations are caused by following reasons: one side of order book is empty, the matching rule cannot be satisfied, and the stop conditions are reached. Therefore, the external main grid needs to further handle them in the balancing operator.

5.7 Comparison

5.7.1 Setup

A phased comparative design is adopted to compare different market mechanisms, pricing mechanisms and bidding strategies in sequence. In each stage, only the target variable is changed while all other experiment conditions remain the same, as shown in **table 14**.

All comparisons were conducted under the same basic conditions, including the same 20 household data set and the annual simulation time. Moreover, the shout prices are all subject to tariff bounds, the interval [FiT, ToU]:

$$FiT \leq p_{bid}, \qquad p_{ask} \leq ToU \tag{25}$$

where FiT is the feed-in tariff and the ToU is the time-of-use tariff. It can ensure that sellers did not submit asks below the export tariff and buyers did not submit bids above the import tariff

Also, for relative fairness in strategy comparison, the shout prices used by the static strategy are the same as the initial prices of ZIP **(21)(22)**. The main initial parameter settings for ZIP and AA are shown in **table 15**. ZIP parameters are initialized randomly within a range; the AA parameters are set to the default values.

Comparison stage	Comparison setting	Evaluation focus
Stage 1 market mechanism	Households use the Static strategy for bidding, while each market mechanism adopts its own typical pricing rule.	The order-clearing capability of the market: the trading volume and the trade numbers
Stage 2 pricing mechanism	Households use the Static strategy for bidding, and the market mechanism selected from Stage 1 is adopted.	The effect on households: buyer–seller benefits distribution
Stage 3 bidding strategy	The market and pricing mechanism selected from Stage 1 and Stage 2 respectively, are adopted.	The effect on entire system: the net savings

Table 14 The three phased comparison

Parameter	Strategy	Setting	Parameter	Strategy	Setting
r, initial aggressiveness	AA	0.0	μ, profit margin	ZIP	U(0.05,0.35)
θ_{init}, curvature parameter	AA	-4.0	β, learning rate	ZIP	U(0.1,0.5)
β_r, aggressiveness update rate	AA	0.4	γ, momentum coefficient	ZIP	U(0,0.1)
β_θ, update rate	AA	0.4	c, perturbation	ZIP	U(0.0, 0.03)
ρ, decay factor for recent trades	AA	0.9			

Table 15 Initial parameters setting

5.7.2 Results and discussion

The mechanism comparison shows that these three have the similar matched/unmatched volume and number of trades. As shown in **Fig. 11**, CDA achieves the highest annual matched volume and trade numbers, while the unmatched volume is the lowest, but the differences in performance are relatively small. Therefore, not only the clearing structure of CDA is more consist with this project, but also the numerical results.

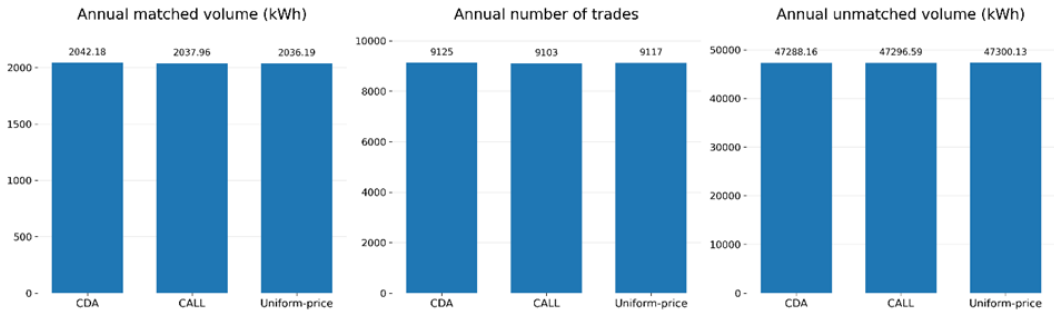


Fig. 11 Annual summary for market mechanism comparison

The pricing mechanism comparison shows the buyer savings and seller gains which shown in **Fig. 12**. The results show that the main difference is how pricing mechanisms affect the profit distribution between the buyers and the sellers. The midpoint demonstrates a more balanced profit distribution, £100.11 and £73.57 respectively, which is the closest, without significant favoured either side. Therefore, from the perspective of relative fairness for profit balance, the midpoint rule is more suitable for the price mechanism in this project.

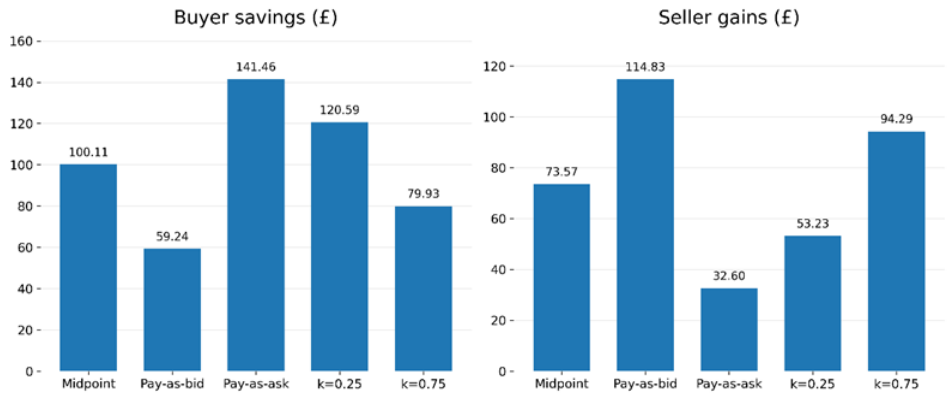


Fig. 12 Annual summary for pricing mechanism

Fig. 13 compares the three bidding strategies in terms of the matched volume, number of trades, savings relative to grid-only trading, and external net cost. All three strategies achieve cost savings compare to trading with the main grid, indicating that the local P2P market can bring certain financial benefits. ZIP and

AA perform better than static, with 2083.72 kWh matched volume and £193.45 savings, suggesting that the adaptive strategies are better to the non-adaptive baseline strategies in general. On the other hand, ZIP and AA perform identically in the trade volume, but ZIP has slightly fewer trade numbers 9313 than AA 9501, shows ZIP achieves the same trading volume with fewer trade numbers, which is more efficient in operational level.

ZIP is a light smart strategy whereas AA is more advanced, but their performance is almost the same, with no clear advantage for AA. Thus, ZIP is a more practical choice for the current system due to its lower complexity and risks. In a larger volatile scenario, AA's long-term learning may become valuable.

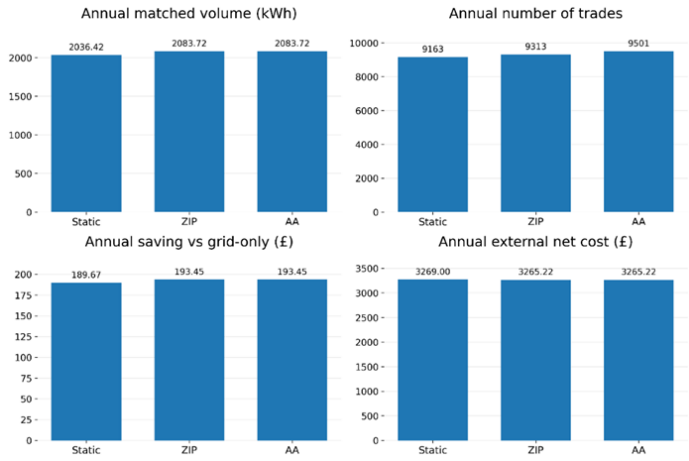


Fig. 13 Annual summary for bidding strategy comparison

The above comparison has determined that the system adopts CDA, midpoint rule, and ZIP combination. Although the results vary similar, this group performed relatively better under the testing conditions.

6 Balancing Operator (YD)

This chapter is mainly responsible for dealing with the imbalance problem in the entire system. Mainly targeting orders that have not been successfully matched after the market clearing, as well as the deviation between actual trading electricity and forecasted trading electricity during actual operation.

6.1 Overview

In the system, the households submit import and export energy quantities based on predictions, and the market operator matches them through the CDA mechanism for each timeslot. Due to CDA only being executed when the buying and selling quotes meet the matching criteria, some orders may still not be matched after the end of the trading period. The existing CDA-based P2P research has also pointed out that there may be unmatched orders after each market period and further liquidation is needed [25]. In addition, during actual operation, the actual operational results of households may differ from the predicted results, resulting in deviations.

These two remaining imbalances are handled by the Balancing Operator. Since the stages and settlement meanings of the unmatched orders and the deviations which in actual operation are different, this chapter adopts a two-stage settlement framework to handle them separately:

- **Stage 1:** After the market clearing, the orders that have not been successfully matched in the market

will be traded with the main grid.

- **Stage 2:** After receiving actual orders data, the deviations between the actual orders data and the already traded orders will be handled. The deviations will be prioritized for internal matching within the community, and the remaining deviations will be handled by the main grid as a fallback.

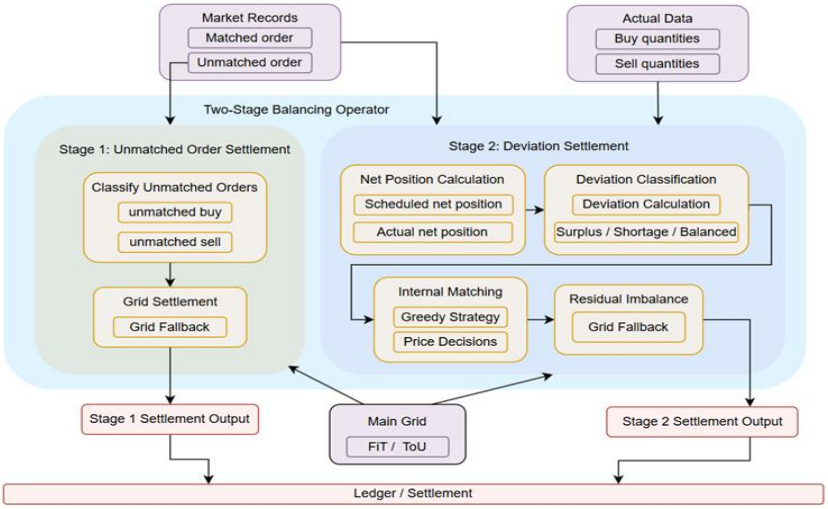


Fig. 14 Overview of Balancing

6.2 Stage 1: Unmatched Order Settlement

In Stage 1, all unmatched buy and sell orders remaining after market clearing are settled directly with the main grid. This stage handles the predicted energy quantities that were not cleared in the local P2P market. For the scheduled energy quantity that has not been successfully matched in the market operator, as it is still the expected buying and selling demand submitted by the household, this part of the quantity needs to be directly settled with the grid, and the settlement amount is defined as:

$$C1_{i,t} = P_t^{FiT} Q_{i,t}^{sell,unmat} - P_t^{ToU} Q_{i,t}^{buy,unmat} \tag{26}$$

where $C1_{i,t}$ is the Stage 1 unmatched order settlement amount for household i in timeslot t, $Q_{i,t}^{sell,unmat}$is the unmatched sell quantity, $Q_{i,t}^{buy,unmat}$ is the unmatched buy quantity. In this report, P_t^{FiT} refers to the price at which the grid acquires electricity from households during timeslot t. Different from the traditional Feed-in Tariff set by the government [30]. This report sets P_t^{FiT} as the grid acquisition price that varies over time. Therefore, it can be expressed as the compensation price obtained by the household when selling electricity to the main grid. For P_t^{ToU}, it represents the Time-of-use price, which is the price at which a household purchases electricity from the main grid [31].

Under this definition, the settlement amount is positive when electricity is sold to the grid and negative when electricity is purchased from the grid.

After Stage 1 is completed, all unmatched orders are traded. After receiving the actual data from the household, Stage 2 will be carried out to handle the deviation between the actual and scheduled data.

6.3 Stage 2: Deviation Settlement

To determine the settlement energy quantity for Stage 2, it is first necessary to calculate the scheduled net position and actual net position of each household at each timeslot. The scheduled net position reflects the scheduled energy position of the household that has been formed after market clearing and the completion of Stage 1 settlement; The actual net position is obtained based on the actual operational data of the

household. By comparing them, it is possible to identify the deviations of each household and further determine whether they are in a state of surplus, shortage, or balanced.

6.3.1. Scheduled and Actual Net Positions

For each household in each timeslot, the scheduled orders are determined from the market records, including successfully matched orders in the local market and unmatched orders settled with the main grid in Stage 1. The total scheduled buy quantities $Q_{i,t}^{sche,buy}$ and the total scheduled sell quantities $Q_{i,t}^{sche,sell}$ are defined as:

$$Q_{i,t}^{sche,buy} = Q_{i,t}^{buy,mat} + Q_{i,t}^{buy,unmat} \quad (27)$$

$$Q_{i,t}^{sche,sell} = Q_{i,t}^{sell,mat} + Q_{i,t}^{sell,unmat} \quad (28)$$

where $Q_{i,t}^{buy,mat}$ and $Q_{i,t}^{sell,mat}$ denote the matched buy and sell quantities, and $Q_{i,t}^{buy,unmat}$ and $Q_{i,t}^{sell,unmat}$ denote the corresponding unmatched quantities.

In each timeslot, each household is assumed to only participate in one side of the market, whether as a buyer or a seller. Therefore, for a given household and timeslot, the scheduled buy and sell quantities cannot be positive at the same time. The net position formulas represent the buyer and seller status, as follows:

$$Q_{i,t}^{sche} = Q_{i,t}^{sche,sell} - Q_{i,t}^{sche,buy} \quad (29)$$

$$Q_{i,t}^{act} = Q_{i,t}^{act,sell} - Q_{i,t}^{act,buy} \quad (30)$$

where $Q_{i,t}^{sche}$ and $Q_{i,t}^{act}$ denote the scheduled and actual net position of household i in timeslot t,respectively. $Q_{i,t}^{act,sell}$ and $Q_{i,t}^{act,buy}$ denote the corresponding actual sell and buy quantities, respectively. A positive value indicates need to sell the quantities, whereas a negative value indicates need to buy the quantities.

6.3.2. Deviation Definition

The deviation $\Delta_{i,t}$ is defined as the difference between the actual and scheduled net positions given in **(29) and (30):**

$$\Delta_{i,t} = Q_{i,t}^{act} - Q_{i,t}^{sche} \quad (31)$$

where $\Delta_{i,t}$ denotes the deviation of household i in timeslot t.

The sign of $\Delta_{i,t}$ determines the types of deviation, as summarised in **Table 16**.

Condition	Deviation Type	Interpretation
$\Delta_{i,t} > 0$	Surplus	The household generated more electricity than scheduled
$\Delta_{i,t} < 0$	Shortage	The household consumed more electricity than scheduled
$\Delta_{i,t} = 0$	Balanced	Actual quantities are the same as scheduled quantities

Table 16 Types of deviation

The absolute magnitude of deviation, denoted $|\Delta_{i,t}|$, represents the number of quantities that remains to be settled in Stage 2.

6.3.3. Internal Matching Price

When resolving deviations internally through P2P transactions, a fair price is required. The internal matching price P_t^{int} is defined as the average value of P_t^{FiT} and P_t^{ToU}:

$$P_t^{int} = \frac{P_t^{FiT} + P_t^{ToU}}{2} \quad (32)$$

Designing prices within the range of P_t^{FiT} and P_t^{ToU} is to ensure that sellers can sell at a higher price than P_t^{FiT}, while buyers can buy at a lower price than P_t^{ToU}. Due to the pricing of internal matching in the Balancing Operator having no impact on the internal matching quantities, it only serves as a settlement. Therefore, this report adopts the mid-market rate (MMR) pricing method, setting the internal transaction price as the average of the grid purchase price and sale price. Related P2P energy trading research has adopted MMR as a commonly used internal transaction pricing mechanism in community microgrids, used to determine fair settlement prices between buyer's grid purchasing prices and seller's feed-in revenue [32].

6.3.4 Stage 2 Settlement Amount

After determining the deviation of each household, the deviation will first be internally matched among participating households, as detailed in Section 6.4. If there are still residual deviations after internal matching, it will be settled with the main grid at P_t^{FiT} and P_t^{ToU}. The Stage 2 settlement amount is therefore defined as:

$$C2_{i,t} = (P_t^{int} Q_{i,t}^{int,sell} - P_t^{int} Q_{i,t}^{int,buy}) + (P_t^{FiT} Q_{i,t}^{grid,sell} - P_t^{ToU} Q_{i,t}^{grid,buy}) \quad (\mathbf{33})$$

where $C2_{i,t}$ is the Stage 2 deviation settlement amount, $Q_{i,t}^{int,sell}$ and $Q_{i,t}^{int,buy}$ are the quantities settled through internal matching, $Q_{i,t}^{grid,sell}$ and $Q_{i,t}^{grid,buy}$ are the residual quantities settled with the main grid.

6.4 Internal Matching

In Stage 2, the balancing operator needs to settle the deviation between scheduled and actual energy quantities. Since such deviations only become observable when actual operational data are received, they cannot be resolved through the original market matching process, which is completed based on predicted orders. Based on this, this part introduces an internal matching mechanism within the community in Stage 2, to prioritize absorbing the deviations within the community before settling with the main grid.

The introduction of internal matching has three main purposes. Firstly, prioritize achieving internal balance within the community and reducing the remaining deviation of electricity that ultimately needs to be settled by the main grid. Related studies have shown that aggregated transactions between consumers and prosumers for local trading can reduce imbalances and potential penalty costs [33]. Secondly, settling part of the deviation electricity at the internal matching price in **(31)** improves household economic benefits, as it is more favourable than direct grid settlement. Thirdly, improve the utilization of flexibility within the community and reduce reliance on external main grid fallback.

6.4.1 Rationale for choosing the Greedy Strategy

In stage 2, different rule-based matching methods can be used to allocate the deviation energy of internal matching. In [34], it is mentioned that under the French collective self-consumption (CSC) rule, the allocation method includes distributing locally shareable electricity based on the proportion of consumer electricity demand. This method allocates electricity based on the proportion of each consumer's electricity consumption to the total electricity consumption, with high transparency and fairness.

However, considering this project, due to the large size of the community, which includes 100 households, adopting a proportional allocation method directly may result in overly fragmented settlement results. For example, 1 kWh of remaining electricity may be proportionally split among many households, with each

household receiving a smaller share of electricity, thereby increasing the granularity of accounting records and increasing the complexity of subsequent billing and settlement audits.

Therefore, this project chooses to use Greedy Strategy as the internal matching strategy for the second stage of balancing. Previous studies have shown that greedy algorithms can prioritize the selection of suitable energy suppliers based on the power supply capabilities of producers and consumers [35]. In this project, the strategy is used to match household surplus and shortage electricity within a shorter settlement period, helping more household deviations to be fully absorbed and recording corresponding bilateral transactions.

6.4.2 Matching Procedure

In each timeslot, the system first divides households into surplus side and shortage side based on the direction of deviation **shows on Table 16**. And based on the Greedy Strategy, use the current remaining deviation of each household as the sorting and matching basis for internal matching. The internal matching will not end until there is no deviation on one side.

In each round of matching, the households on the surplus side and the shortage side will be sorted by ascending order, according to the current remaining deviation. Subsequently, the system selects the households with the smallest remaining deviation from both sides to form active matching groups for the current round. The matching quantity in each round is determined by the smaller of the total surplus volume in the active surplus group and the total shortage volume in the active shortage group, as shown below:

$$Q_{g,t}^{match} = \min\left(\sum S_{g,t}, \sum H_{g,t}\right) \quad (34)$$

where $Q_{g,t}^{\text{match}}$ is the total matchable quantities for group g in timeslot t, $\sum S_{g,t}$ is the total surplus quantities in the active surplus group, and $\sum H_{g,t}$ is the total shortage quantities in the active shortage group. Under this rule, at least one side is fully absorbed in each matching round.

After completing the allocation round, the system will record the corresponding internal transactions and update the remaining deviation of all participating households. For households whose remaining deviation has been reduced to zero, remove them from the matching pool. The remaining households will continue to participate in the next round of matching. The above process will continue to repeat until there are no more surplus and shortage within the current timeslot that can be further internally hedged. The deviation electricity that has not yet been internally absorbed will be settled by the main grid.

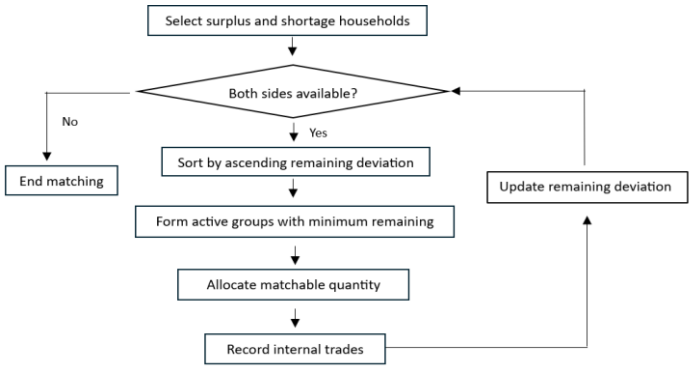


Fig. 15 Greedy Internal Matching

6.5 Implementation and Comparison

6.5.1 Input Data

The balancing mechanism uses three types of input data: market records, actual data, and tariff data.

- Market records provide matched buy and sell quantities and unmatched buy and sell quantities for each household under each timeslot, used to determine scheduled positions and settlement electricity for Stage 1.
- Actual data provide actual buy and sell quantities for calculating the deviation between actual and scheduled net positions in Stage 2.
- Tariff data provides import and export prices for internal matching price calculation and main grid settlement.

All data is aligned by household and timeslot before executing the settlement procedure.

6.5.2 Implementation Workflow

The balancing mechanism was applied to each timeslot in sequence. For each timeslot, market records, actual data, and tariff data were first aligned by household and timeslot. The scheduled net position and actual net position were then calculated using **(29) and (30)**.

After the actual data were received, the deviation of each household was calculated by comparing the scheduled net position with the actual net position. Afterwards, the settlement process follows the two-stage settlement framework.

In Stage 1, all unmatched buy and unmatched sell quantities are settled directly with the main grid. In Stage 2, all deviations are first settled within the community through internal matching, and the remaining portion is then settled by the main grid.

Finally, two sets of results were generated. Stage 1 is mainly outputs unmatched quantities and the settlement amount obtained based on **(26)**. And Stage 2 outputs the settlement amount obtained through internal matching and grid fallback, based on **(33)**.

6.5.3 Comparison Setup

This experiment included two sets of comparisons. The first group is used to evaluate the role of internal matching mechanisms in Stage 2, while the second group is used to compare the impact of different internal matching rules.

All experiments used the same market records and actual operational data from 20 households in the same year, tariff data, hourly settlement intervals, and research cycles to ensure comparability of results.

Comparison 1: Internal Matching vs Grid-only

To evaluate the effectiveness of internal matching, two scenarios were set up:

- **Scenario A:** The deviations in Stage 2 are first matched within the community, and the remaining surplus or shortage is then settled with the main grid.
- **Scenario B**: All deviations in Stage 2 are directly settled with the main grid without internal matching.

This comparison was used to analyse the impact of internal matching on household settlement cost, grid import/export volume, and local balancing performance.

Comparison 2: Greedy vs Proportional Matching

Based on Scenario A, further compare the two Stage 2 internal matching rules:

- **Scenario A1:** Greedy matching

- **Scenario A2**: Proportional allocation
- In this comparison, the overall settings remain consistent with Scenario A, with the only difference being the internal matching allocation rules used in Stage 2. This experiment was used to evaluate the impact of different matching rules on fully absorbed household cases and residual deviation.

6.6 Results and Discussion

6.6.1 Results of Internal Matching

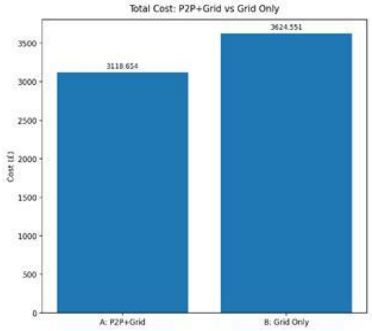


Fig. 16 Total cost

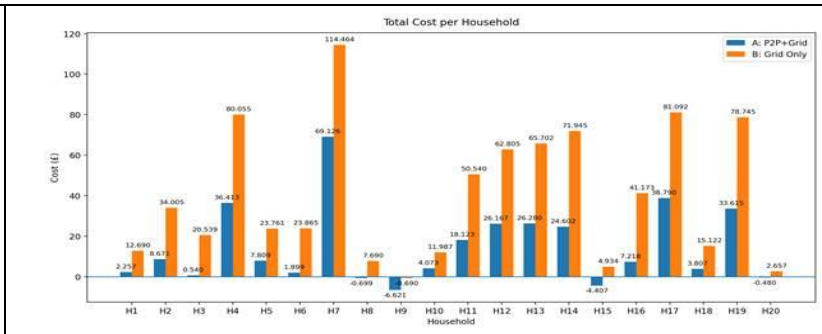


Fig. 17 Household cost: P2P+Grid vs Grid-only

Fig. 16 and **Fig. 17** compare the total cost and household level settlement cost between Scenario A and Scenario B, respectively. These figures indicate that the settlement cost under scenario A is lower than scenario B. At the household level, H9, H15, and H20 show negative settlement costs under scenario A, indicating that these households have achieved net economic benefits. Overall, the results indicate that internal matching can reduce the cost of household to some extent and may even earn a certain profit.

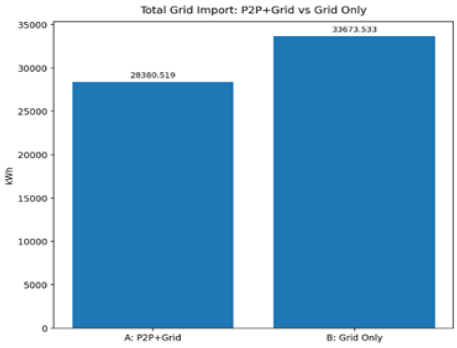


Fig. 18: Total Grid Import Volume

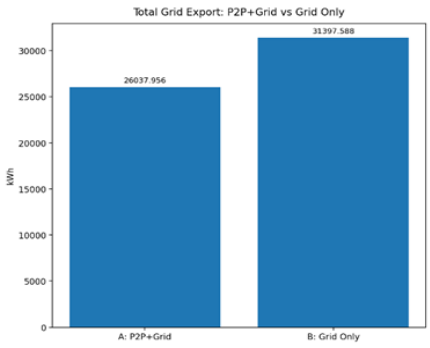


Fig. 19: Total Grid Export Volume

Fig. 18 and Fig. 19 show that the total grid import and export of Scenario A are lower than those of Scenario B. This indicates that internal matching can absorb some deviations within the community, thereby reducing the settlement volume with the main grid and improving local balancing performance.

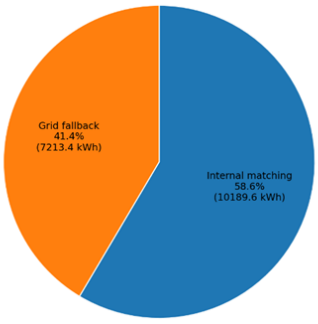


Fig. 20：Stage 2 Settlement Share: Internal Matching vs Grid Fallback

Fig. 20 shows that 57.0% (1018.96 kWh) of deviations in Stage 2 were settled through internal matching, while 43.0% (767.64 kWh) were settled through grid fallback. This indicates that internal matching has absorbed more than half of the deviation, reducing dependence on the main grid.

6.6.2 Results of Matching Rules

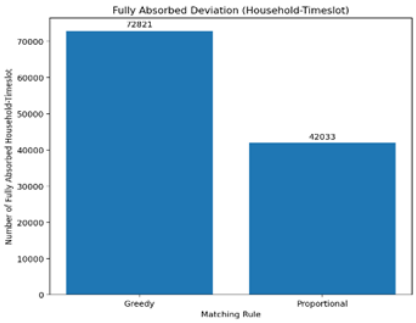


Fig. 21: Fully Absorbed Deviation (Household-Timeslot)

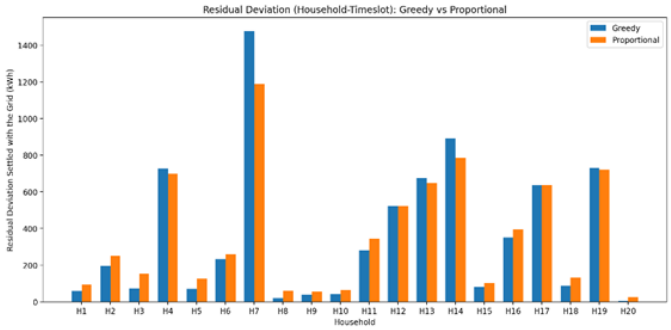


Fig. 22: Residual Deviation (Household-Timeslot)

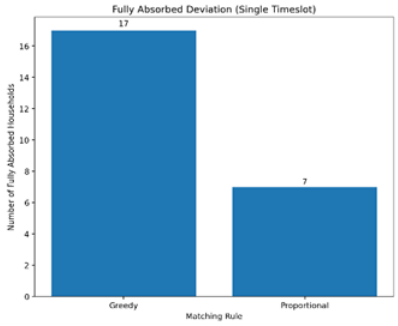


Fig. 23: Fully Absorbed Deviation (Single Timeslot)

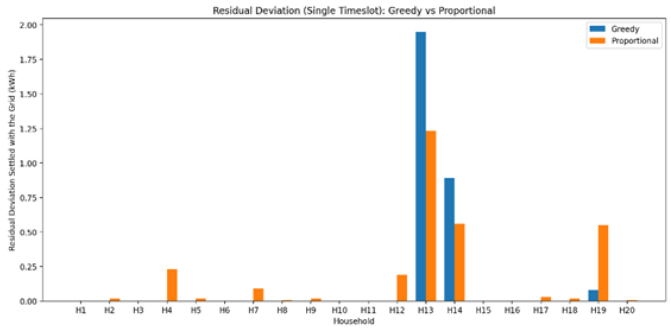


Fig. 24: Residual Deviation (Single Timeslot)

Fig. 21 and **Fig. 22** show the overall comparison results based on household timeslot-records, where one household-timeslot record represents the settlement record of a household in a certain timeslot. **Fig. 23** and **Fig. 24** show the performance of a greedy matching and proportional allocation in a timeslot.

Fig. 21 and **Fig. 23** show that whether it is a timeslot or a full year, compared to proportional allocation, greedy matching can fully absorb more deviations from one household, indicating that greedy matching can enable more households to no longer rely on the main grid during the corresponding timeslot. **Fig. 22** and **Fig. 24** show that greed does not necessarily minimize the residual deviation of each household like proportional allocation does. It concentrates the remaining deviation a few households. The deviations of most households have been completely absorbed internally. Therefore, greedy matching is more helpful in reducing residual fragmentation, improving the clarity and traceability of settlement results.

6.6.3 Discussion

Overall, the experimental results comparing with and without internal matching further show that internal matching plays an important role in the second stage deviation settlement. Compared to settling directly with the main grid, internal matching can absorb a portion of excess and shortage electricity within the community. This can not only fully utilize the resources within the community at a lower cost or high income than the trade with main grid but also reduce dependence on the main grid.

In terms of comparing matching rules, greedy matching is more suitable for the goals of this project than proportional allocation. It can generate more fully absorbed household-timeslot records, reduce the number

of households trading with the main grid, and enable more households to experience the benefits of smart microgrids. It can also reduce the situation where residual deviations are dispersed to multiple households simultaneously, facilitating the recording of settlement results and making it easier to track.

7 Project Integration and Deployment (JL)

7.1 Introduction

This section explains how the separate sectors of this smart microgrid prototype were integrated into one working end-to-end project. This provides the technical framework that allows those sectors to function together as one complete system. It focuses on explaining how the different sectors are:

- Packaged (Using Docker and a requirement file)
- Assembled (Using Docker Compose)
- Connected (Using FastAPI and HTTP)
- Coordinated (Using Orchestrator)
- Stored (Using Database)
- Visualised (Using Dashboard)
- Expansion (Tailscale)

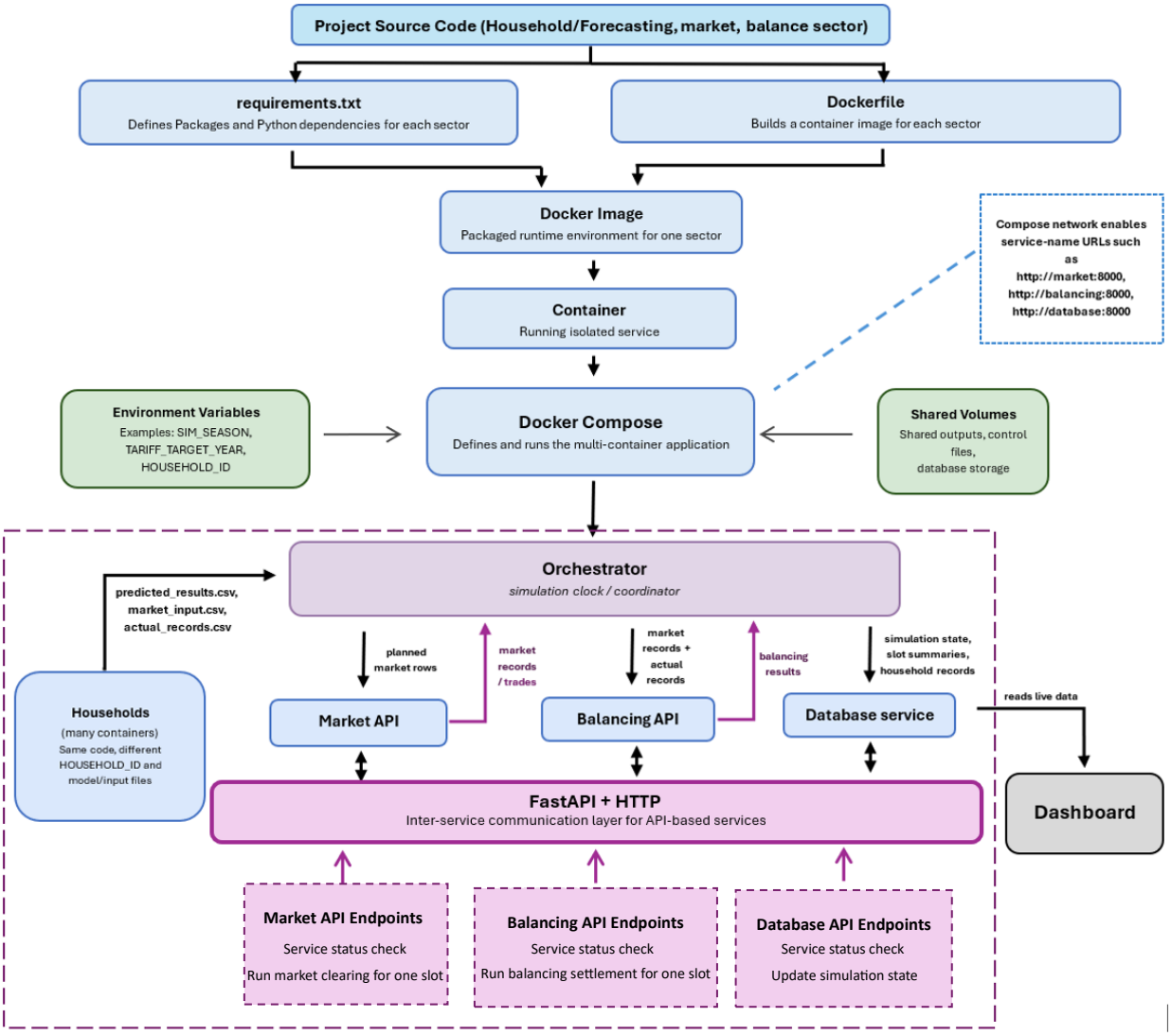


Fig. 25 (System Integration Architecture)

7.2 Execution foundation

7.2.1 Why Docker was chosen

Docker was chosen because the project required more than just Python dependency isolation. Although each sector can be run in a Python virtual environment individually, it does not package system libraries, runtime configuration, entry point or service startup. By contrast, Docker packages each sector together with its runtime environment inside a container image and runs the image as an isolated process that shares the host kernel. Docker's own documentation describes it as streamlining the development lifecycle, allowing developers to work in standardised environments using local containers that provide their applications and services. [36]This makes the system more reproducible and less dependent.

7.2.2 Dependency control across sectors

Docker is perfectly suited to this project, as each Docker's requirement file separates operational responsibilities across sectors while enabling a continuous workflow. For example, the market, balancing and database relied on packages such as FastAPI, uvicorn, Pydantic, pandas and NumPy, while the household sector depends on machine-learning packages with more tightly controlled versions. This is especially important for the forecasting model, as TensorFlow and Keras are sensitive to version mismatches. Using Docker avoids version mismatches and enables the separation of these dependencies for each sector without affecting the rest of the system.

7.2.3 Docker as a sector execution model

In this project, Docker is not only used for its convenience or packaging, but it is also used because it simulates the smart microgrid architecture behaviour. A smart microgrid is not a single program, it is a system made up of distinct sectors. It allows each of these sectors to run as its own container with its own runtime identity, this separation is valuable both technically and conceptually. Technically, as discussed above, Docker avoids forcing all the sectors into one shared software environment. Conceptually, it reflects the structure of a real system being modelled, where different functions operate as separate units but still interact within one overall platform.

7.2.4 Role of requirements files and Dockerfiles

Within this project, the role of the requirements files is to determine which Python dependencies or packages are needed, whereas the Dockerfile determines how the sector is built and what process is executed when the container starts. Together, they provide a clean execution environment for each sector of the system.

7.3 Docker Compose as the System assembly layer

7.3.1 The selection of Dock Compose

If Docker defines how each sector is packaged and executed individually, Docker Compose defines how the full smart microgrid platform is assembled and run as one application. As Docker Compose is a natural extension of Docker, it is designed for running multiple containers through a single YAML configuration file. This is ideal for this project as it simplifies the management of different services and volumes in one place, with all the services being created and started together in a controlled way.

7.3.2 Multi-service startup and coordination

Docker documents state that this command builds, recreates, starts, and attaches to the service container while starting linked services when required. [37]This is particularly valuable because sectors are

interdependent, for example, the orchestrator depends on the market, balancing and database service, while the dashboard depends solely on the database. Docker Compose, therefore acts as the layer that brings these sectors under control and in an organised way rather than a manual start-up order.

7.3.3 Household duplication and scaling

A major advantage of Docker Compose in this project is that it turns one household's implementation into a population of households, each one of them have different behaviours without duplicating the household source code. The household services inherit the same common build structure, but each one is given a different Household ID and a fine-tuned prediction model, allowing the same codebase to be executed repeatedly with different identities and inputs. This makes it possible to simulate a large household community while preserving one shared household codebase.

7.3.4 Centralised experiment configuration

Docker Compose also act as the main configuration layer for the experiment. Instead of editing Python source files whenever a scenario changes, the environment variables in the Compose file allow different simulation parameters to pass through, such as seasons and tariff target year shows that key scenario assumptions can be changed centrally at the deployment level rather than being hard-coded into each sector. In this project, different seasons will change the amount of the PV surplus. This feature enables repeated testing under different operating conditions when combining all the sectors together.

7.3.5 Health checks and startup reliability

Docker Compose health checks are also an important feature that allows the system to distinguish between a container that has just simply started and a service that is genuinely ready to operate. In a multi-service project, this distinction matters as container startup does not automatically mean that the application inside the container is already listening or ready to serve other sectors.

In this project, the database health check is especially important as the database stores system updates, slot summaries and household-level outputs. The dashboard depends on it for visualisation health check, therefore ensuring that the database is genuinely ready for the dashboard to read and display its live values. Therefore, the health check points in Docker Compose repeatedly verify each sector through its health endpoint until it is truly ready, enabling a stable startup for the whole system.

7.3.6 Larger-scale deployment across multiple machines

A single machine can host many containers, but one laptop still has hardware limits as running multiple households can be computational heavy, but because the household population is defined in a structured Compose model, it becomes much easier to increase or reduce the number of households, this is particularly helpful later when scaling the project to larger community sizes such as 100 households..

Therefore, Docker Compose is not just a development tool, but also the assembly and scaling layer of this smart microgrid project.

7.4 Tailscale as the secure Multi-Host Deployment Layer

One of the main limitations of this project was not the software architecture itself, but the hardware capacity of a single laptop. Although Docker Compose made it possible to define many household containers in a structured and repeatable way, running all household containers on one host machine can be computationally heavy. Each household still had to load its own model with its own time series data and generate its own output files for the market and balancing operator. This meant that a single laptop could

not reliably execute the full 100-household community together, especially with the rest of the system services. The project needs a way to distribute household execution across several physical laptops while keeping the same overall system structure. In this project, Tailscale provided the extension layer that allows workers and the host to communicate securely in a multi-host machine system.

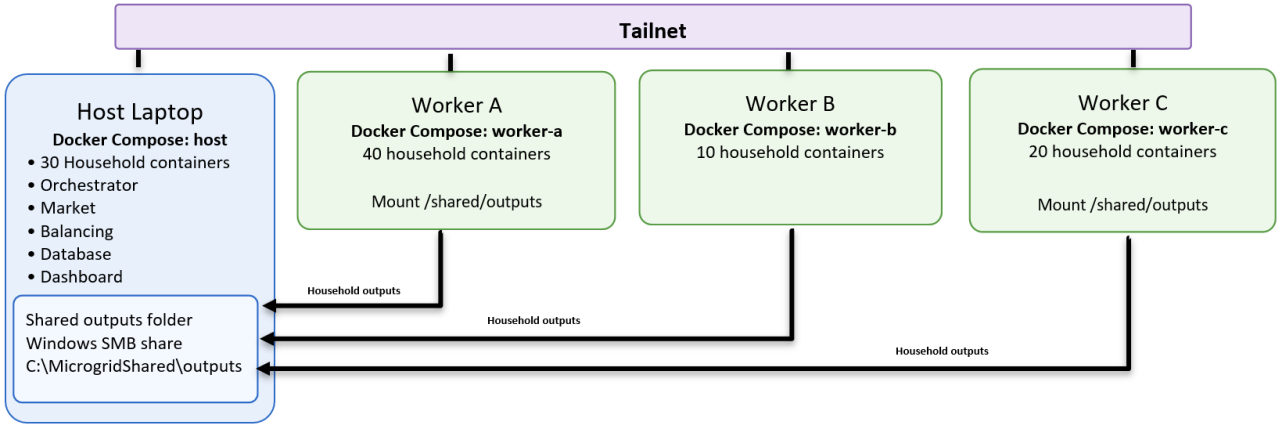


Fig. 26 (Multi-laptop deployment using the Tailscale Tailnet for the 100-household microgrid simulation)

In practice, the host laptop created a shared results folder that acted as the single collection point for all workers and host outputs. The host laptop ran the main system services alongside the 30 households container, while the other worker laptops were responsible for running only batches of household simulation. Each worker then ran their assigned households and saved the generated household results in that shared location. The shared file on the host machine was made available to all workers through the Tailnet. Notice, the Windows share on the host was mounted inside Linux as a normal project folder using SMB/CIFS protocol while Tailscale made the host reachable over Tailnet.

The project chose Tailscale not only for its connectivity convenience but also for the privacy and security it provides. It creates a private Tailnet between host machines and uses WireGuard-based encryption for communication. This is important because it means that the worker laptop and the host machine can exchange data over a private, encrypted network rather than relying on ordinary local network addressing. In a real microgrid, household community data would be sensitive in real deployment. Information such as household demand profiles, battery behaviour, and trading activity could reveal patterns of household behaviour and energy use. Tailscale is therefore not only a scaling solution but also a realistic approach that reflects the privacy expectations of a real microgrid community deployment.

7.5 Inter-Service Communication Layer

7.5.1 FastAPI as API layer

FastAPI was selected as the API layer because the project required lightweight communication between independent sectors. This project is organised as a set of separate services that need to exchange data in a clear and structured way. So FastAPI provided a clear communication layer to the system, in other words, this means that a sector can be turned from a normal Python code module into a callable service with defined routes, input structures and output responses.

7.5.2 FastAPI, HTTP/URLs and Docker Compose networking

In this project. FastAPI, HTTP/URLs, and Docker Compose networking should be understood as separate but

complementary parts of the same communication architecture. FastAPI defines the API endpoints, HTTP provides the request-response mechanism, URLs identify where those endpoints can be reached, and Docker Compose networking allows the containers to discover one another by service name. Together, these components form the communication structure through which the system's sectors interact.

With FastAPI, different sectors can behave as independent services while still participating in one overall simulation pipeline. For example, the orchestrator does not directly import and execute all the sector logic inside one Python process. Instead. It sends HTTP requests to sector APIs. This means that the market sector can expose a route to receive one slot of planned household trades, the balancing sector can expose its own route to process market outputs together with the actual records, and then the database sector can expose routes such as a simulation state. Slot summaries and household records to store and provide simulation outputs for the Dashboard. In this way, service routes can be inspected and tested more easily. This is particularly important in a project where many sectors need to interact repeatedly. The FastAPI communication layer will help reduce ambiguity and allow the interactions between sectors to be understandable and robust as the system grows.

7.6 Central control and timing layer

7.6.1 Role of the orchestrator

The orchestrator acts as the simulation clock and coordination layer of this project. While the household, market, and balancing sectors each perform their own functions, the orchestrator combines them and enforces a slot-by-slot workflow so that the entire platform behaves as a single, end-to-end system. It controls the order in which each sector is executed, ensures that the same simulation time step is used across the system, and passes the correct data from one service to the next. Therefore, the orchestrator is not simply another sector but the central control point of the project.

7.6.2 Startup configuration, live control & readiness check

At the start of execution, the orchestrator reads the runtime configuration. This includes participating households, the shared output location, the service address of the market, the balancing and database sector, and it also reads the simulation speed and runtime control settings. This makes the orchestrator flexible and consistent with the wider containerised deployment design, because it no longer depends on fixed local settings. Instead, it can adapt to different runs while preserving the same overall control logic. In addition, the orchestrator reads the external control settings during execution, which allows the simulation to be paused and the step speed to be adjusted while the system is running.

Before the main simulation loop begins, the orchestrator performs multiple validation and readiness tasks that are important to the system. As shown in the **Integration and deployment architecture diagram**, it waits until the market, balancing and database service are reachable, and waits until the household has produced the required files in the shared output directory. These files contain predicted household results, planned market inputs, and actual trading records. After loading these files, the orchestrator will validate that all three files exist for every household, that each dataset contains a time column, that the row counts are consistent, and that the timestamps are aligned across all household records. This prevents the simulation from continuing with inconsistent time-series inputs, which would otherwise undermine the validity of later market and balancing results.

7.6.3 Writing outputs to the database

Once the data has been validated, the orchestrator becomes the active control centre of the simulation and begins the slot-by-slot execution. For each simulation step, it gathers the planned household import and export position together with the corresponding actual buy and sell values. It will first send the planned market positions to the market service. After receiving the market response, the orchestrator will then send the market settlement record, together with the actual household trading records, to the balancing service. And when the balancing is complete, the orchestrator combines the outputs of both sectors and produces the main system-level metrics that are later written to the database and displayed in the dashboard. This means that the orchestrator is responsible not only for sequencing the services but also for generating the summary values for the dashboard interpretations.

7.6.4 System-level calculations

Orchestrators are responsible for important system-level calculations. At the community level, the total peer-to-peer traded energy in each time slot is calculated as:

$$\boldsymbol{P2P_{total} = P2P_{planned} + P2P_{balancing}} \quad \textbf{(35)}$$

This is obtained by combining the planned peer-to-peer energy matched during the market stage for the predicted import/export energy received from the household sector, together with the additional peer-to-peer energy matched from the deviation energy in the balancing operator. In the same way, the total electricity bought from the grid is calculated as the sum of the planned unmatched buy quantity and additional external purchase required after balancing:

$$\boldsymbol{Grid_{bought} = Grid_{planned} + Grid_{balancing}} \quad \textbf{(36)}$$

And using these 2 values, the P2P trade percentage can be calculated as:

$$\boldsymbol{P2P\ trade\ \% = \frac{(P2P_{total})}{P2P_{total} + Grid_{bought}}\ x\ 100} \quad \textbf{(37)}$$

This measures how much of the community's bought electricity was satisfied through internal P2P trading rather than external grid purchase.

7.6.5 Financial comparison with and without P2P

The orchestrator also calculates the net cost with and without peer-to-peer trading. In both cases, the financial structure is based on external grid transactions. The difference lies in whether internal matching has taken place first.

$$\boldsymbol{Net\ System\ Cost_{with\ P2P\ trading} = (Grid\ Buy_{after\ P2P} + P2P\ buy) - (Grid\ Sell_{after\ P2P} + P2P\ sell)} \quad \textbf{(38)}$$

Notice, the price for peer-to-peer buy or sell is equal, so the equation becomes:

$$\boldsymbol{Net\ System\ Cost_{with\ P2P\ trading} = Grid\ Buy_{after\ P2P} - Grid\ Sell_{after\ P2P}} \quad \textbf{(39)}$$

By comparison, the system cost without any internal matching(P2P):

$$\boldsymbol{Net\ System\ Cost_{without\ P2P\ trading} = Grid\ buy - Grid\ sell} \quad \textbf{(40)}$$

The distinction is important. In the without P2P trading case, the community is treated as if there were no internal matching, so all net buying and selling is settled directly with the grid. In the P2P trading case, internal matching happens first, so the remaining grid buy and grid sell values are the external deviation after internal trading. This financial comparison reflects the project's purpose. The aim here is not to show that internal trading can occur, but to evaluate whether internal matching improves the overall

performance of the community in terms of energy usage and costs, provide the basis for assessing whether the P2P system reduces external dependency and improves community-level outcomes.

Overall, the orchestrator contributes not only to sequencing sectors and calculation, but also to the operational behaviour of the whole microgrid system. Without it, the project would remain a collection of individual services. With it, the system becomes a coordinated and executable end-to-end prototype.

7.7 Persistent data storage layer

The database acts as the storage layer of this project. Its role is to store the latest simulation state, slots summary results and household snapshot records that are generated during each simulation step. In the project, the orchestrator writes these outputs to the database after combining the results from the household, market and the balancing sectors, while the dashboard reads the stored values back for live visualisation. In a system where multiple sectors interact, it is important to have the calculation and the presentation separated. The database, therefore, sits between the orchestrator's calculation and the dashboard's presentation, providing one stable source of truth for the system.

SQLite was used as the cross-platform database when making the database, as it provides a lightweight and file-based storage in Python, without having to incur the overhead of running a separate database server. The database needs to be a reliable storage of simulation outputs in a format that can be updated repeatedly and read back easily by the dashboard. In that sense, SQLite make it well-suited to a containerised prototype where it functions as a practical and efficient storage layer that supports the end-to-end operation of this smart microgrid.

7.8 Monitoring and visualisation layer

The dashboard is the monitoring and visualisation layer of this project. For this project, the dashboard will need to display the system metrics, charts, tables and user controls in a form that can be built directly in Python without building a separate front end. In this context, the Streamlit app was selected as it runs from a normal Python script and supports a range of layouts such as chats, tables, and widgets. This makes it suitable for a data-driven interface rather than a conventional website while still including all the necessary features to demonstrate this project's objective. This makes the dashboard a live operational interface for inspecting the behaviour of the smart microgrid prototype.

The Dashboard is not just a static output page. It supports update pausing, adjustable refresh interval, simulation widgets, interactive household selection, and expandable settlement history views. With the features that Streamlit supports, the Dashboard outputs are clear and concise. For example, features like Streamlit Plotly chart enable zoom support and a visible mode bar that allows the graph to be inspected in detail. Streamlit session state and cached data-loading functions allow updates to remain responsive while repeatedly reading live data from the database. Another important design of the dashboard is that it makes the behaviour of individual household agents visible rather than only showing whole-community performances. Streamlit provides interactive widgets and display functions that support this directly, allowing users to choose a specific household for inspection. Together, these features make Streamlit a well-suited framework for this project.

Overall, the dashboard should be understood as more than a visual add-on. In the context of this smart microgrid project, it is an interface for monitoring, inspecting, and demonstrating the operation across the

system in a clear and accessible way. Its functions help turn the integrated architecture into something that can be observed. The code for this implementation can be accessed at [39]

8. Results (JE, GH, JL)

8.1 Forecasting model used in the simulations

The Microgrid simulation used the LSTM64_32 model trained with federated learning followed by fine-tuning, selected in Section 3 based on validation performance and privacy suitability.

After being selected, the stacked LSTM was evaluated on the unseen test set, achieving a mean RMSE of 0.2966 kWh and median RMSE of 0.2689 kWh across the 6-hour prediction horizon. This was then deployed in the 30-household seasonal microgrid simulations

8.2 Seasonal and full-year results for 30 households

The outputs are taken from the whole run dashboard summaries. The table below summarises the main energy and financial indicators for 30 households.

Season /Year	P2P share of supply	Grid energy bought (kWh)	Energy sold to grid (kWh)	Net cost without P2P (£)	Net cost with P2P (£)	Total savings from P2P (£)	Average saving per household (£)
Spring	14.3%	15,888.50	5,628.07	2,935.16	2,678.59	256.57	8.55
Summer	23.5%	5,304.95	12,195.41	87.81	-52.61	140.42	4.68
Autumn	12.4%	22,302.57	1,826.23	4,575.10	4,268.74	306.36	10.21
Winter	11.6%	31,225.65	1,676.75	7,754.46	7,275.99	478.48	15.95
Full year	13.4%	75645.86	21664.75	14440.47	13308.22	1132.25	37.74

Table 17

Table 17 shows that the prototype completed successfully in all 4 seasons, and the peer-to-peer system has helped the community save money in all seasons. The summer results are especially important because the net cost with P2P trading becomes a negative value of -£52.61. This does not indicate an error, this means that the community earned more from electricity exported than it paid for electricity imported from the grid. This is consistent with the summer dashboard output, which shows the lowest Grid energy bought of the four seasons at £5,304.95kWh and the highest energy sold to the grid at 12,195.41kWh with the highest P2P share of supply at 23.5%. These values suggest that strong photovoltaic surplus in summer reduced the community's need to buy from the grid, most of the time, the photovoltaic generated can support their demand while creating substantial export revenue to the grid, which is why the net cost is both much lower than in the other seasons, and with the P2P system, the community is earning money in summer. Across all 4 season runs, the dashboard shows that the peer-to-peer trading was active and that the system consistently produces internal matching rather than only replying to the grid. The P2P share in a 30-household community was 14.3% in spring, 11.6% in winter, 12.4% in autumn and 23.5% in summer. This again indicates that this project prototype has achieve it main aim, it successfully created a measurable internal energy trading within the community, with the strongest internal matching percentage in summer when the photovoltaic surplus was the highest. So in a full year run, the total savings from the P2P system are £1,132.25, which means in the 30-household community, each household can save around £37.74 across the year.

The average pricing output is also consistent with the intended design of the system: in the figure below,

the average P2P price is £0.152/kWh, which lies between the average grid buy price of £0.203/kWh and the average grid sell price of £0.093/kWh. This proves the accuracy of the market, and the balancing operator, since the P2P buy and sell prices are equal, and the P2P price sitting between the Grid buy price and the Grid sell prices indicates that household can buy their energy cheaper through P2P trading and earn more if they sell it into the P2P system.

Fig. 27 shows that the P2P trading is present throughout the run and shows the relationship between external dependence and internal matching across the year directly. From this figure, we can see that the external dependence is visibly higher at the beginning and end of the year, while the middle part of the run shows lower grid import and more favourable conditions for internal matching. This is important as it shows the energy demand changes based on seasons, for example, in the UK, people will use more electricity in Winter when turning on the radiator. This change suggests that the system is responding correctly to the demand usage.

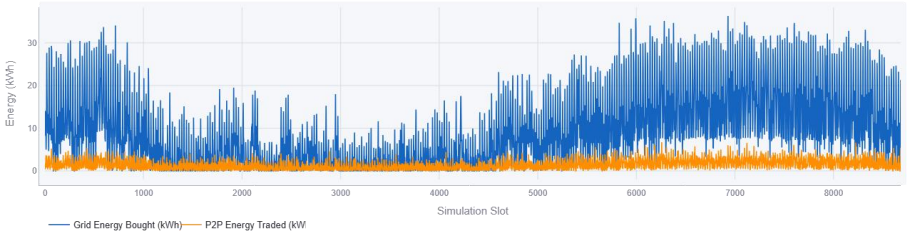


Fig. 27 (Grid Energy Bought vs P2P Energy Traded)

The next important output is the Net External Cost Over Time shown in **Fig. 28**. This figure links directly to the energy behaviour and the financial outcome. The curve is clearly higher at the beginning and end of the run, but becomes much lower, and at times negative, through the middle part of the year. This indicates that during the periods of stronger local generation, the community can reduce or even temporarily earn money by exporting to the grid, which is aligned with the summer savings from the table above. But even so, the high cost periods are still large enough that the final annual net cost remains positive, which means overall the community will not earn money from the grid most of the time.

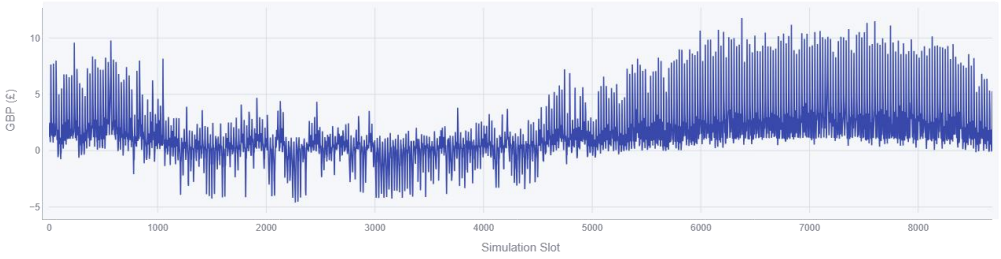


Fig. 28 (Net External Cost Over Time)

8.3 Pricing behaviour and performance for the 100-household community

To test whether this behaviour remained consistent at a larger scale, a 100-household community full-year simulation was also run. Rather than repeating seasonal discussion, this larger case is used for a scalability check. **Table 18** shows that the same overall pattern remains, and the simulation completed successfully alongside active peer-to-peer trading. Most importantly, the net cost with P2P trading was lower than the benchmark case without P2P trading. In the 100-household run, the whole-run net cost fell from $46,429.41 to £42,005.39, giving average savings of £44.24 per household. The P2P share of supply also

increased slightly from 13.4% in the 30-household case to 16% I the 100-household case in a full year. This indicates that the system not only works for a smaller community but also retains the same beneficial behaviour when scaled up.

Metric	30 households	100 households
Grid energy bought (kWh)	75,645.86	237,507.09
Energy sold to grid (kWh)	21,664.75	65,990.75
Total P2P energy traded (kWh)	11,691.29	45,379.01
P2P share of supply	13.4%	16.0%
Net cost without P2P (£)	14,440.47	46,429.41
Net cost with P2P (£)	13,308.22	42,005.39
Total savings from P2P (£)	1,132.25	4,424.02
Average saving per household (£)	37.74	44.24

Table 18: Full-year comparison between 30-household and 100-household runs

Overall, the full-year results demonstrate that the system is logically correct and achieved its main objective as an end-to-end smart microgrid system. The results also show that performance is not constant across the year but changes with operating conditions such as different seasons. This exact kind of behaviour is expected from a realistic community energy system. This prototype can therefore be a useful platform for evaluating peer-to-peer energy trading performance for a household community.

9 Limitations and Further Research (Shared)

Although experimental design and implementation were developed to be as thorough as possible within the scope of the project, several limitations remain. These limitations relate to all modules and provide guidance for future research.

9.1 Forecasting module

The forecasting study focused on lightweight and literature aligned models rather than an exhaustive architecture search. Model selection and early stopping were both based on the validation set, so the reported performance estimates were likely to be optimistic. In the federated setting, although privacy was preserved with respect to raw data, stronger privacy measures such as differential privacy and secure aggregation [7] were not implemented, potentially leading to information being gained from gradient updates [12] [13]. Additionally, hardware constraints limited the scope of federated experiments, particularly regarding communication rounds, participation rates and clustered FL. Finally, the limited-data test used a relatively small household cohort, so its findings should be interpreted as a relative comparison rather than representative performance across the population.

Further research could increase the architectures used, including more complex transformer/ combined models, implementing stronger privacy preserving federated methods and evaluation on a larger number of households.

9.2 Household agent

As discussed above, using average demand to estimate household size can be not rigorous. With the method used it can still cause unrealistic parameter matching which can lead to small errors in results. For the approach for battery sizing, it does not capture differences in peak demand, daily load shape or real

household purchasing decisions. For PV, it does not choose the size based on actual household rooftop area or economic choices because of the limitations of the data given.

In this project, a design choice of prioritising exporting to the grid rather than charging the battery when there is a surplus is made because we want to maximise P2P trading. However, in practice, it's difficult to assume individual decision-making towards energy usage. So, it is possible in real life that many households will choose to fully charge their battery rather than sell when they have enough.

9.3 Market Operator

The market operator constructed a lightweight market operation framework. Due to the adoption of a community-based P2P structure, there has some privacy issue [2]. Also, the experiment is based on fixed conditions, so the results are the relative comparisons, which cannot represent all community market. In addition, market clearing only focuses on the financial-level matching, without considering the actual physical factors such as network losses, grid usage fees[38].

Future research can explore more intelligent bidding strategies, such as the full version of AA, GDX [28], or game theory [39], which can perform better in larger-scale and more dynamic trading environments. Additionally, privacy protection can be further enhanced through blockchain and smart contracts [40].

9.4 Balancing Operator

In the balancing operator, the effect of internal matching is mainly reflected in the distribution of deviation electricity surplus and shortage within the same timeslot. If most households are in the same type of deviation, the remaining imbalance still needs to be mainly covered by the main grid.

The internal matching adopts a greedy strategy, which can allow more households' deviations to be completely absorbed within the community and ensure calculation efficiency. The settlement result is clear and easy to track. However, its outcome is not the optimal solution for all matching combinations. In the future, more advanced internal matching mechanisms can be further studied based on their applicability to large communities and maintaining the clarity of the settlement.

9.5 Visualisation and Deployment Framework

The dashboard and deployment framework were developed to support practical system execution and demonstration. While the dashboard does show the live updates and results effectively, it does not include more advanced analytical features such as scenario comparison or richer filtering. In addition, the multi-laptop deployment still relied on correct shared folder mounting and manual startup order. This makes the workflow more fragile than a fully automated system, while the host laptop remains a central dependency for shared results and core services.

On a visualisation-wise, the dashboard can have a stronger analysis function, such as automated reporting and comparison tools. The deployment framework could also be strengthened by implementing a more robust and fault-tolerant multi-laptops set up with less manual configuration, which will make the multi-laptop deployment have better automation.

10 Conclusion

The results suggest that the project successfully achieved its main aim of developing a working smart microgrid emulation framework that integrates forecasting, household operation, peer-to-peer trading, balancing and system monitoring within one working pipeline. Across the full run, the system recorded measurable peer-to-peer traded energy and P2P share percentage, indicating that the local market and balancing stages were functioning as intended within the smart microgrid system.

The financial results also indicate that this project was able to deliver practical value beyond technical integration. The whole-run summary shows that the total household net cost was lower with P2P trading than in the non-P2P case, leading to savings across the whole community, which supports the idea that local trading is beneficial for both buyers and sellers by allowing household to buy energy below the grid import price and sell surplus energy above the grid export price. The overall findings suggest that integrating forecasting, household energy management and P2P trading can improve household cost outcomes, while moving the community towards a more intelligent local energy system.

References

[1] R. Balakrishnan and V. Geetha, “Review on home energy management system,” *Materials Today: Proceedings,* vol. 47, no. 1, pp. Pages 144-150, 2021.

[2] Z. Tanis, A. Durusu and N. Altintas, “A Comprehensive Review on Peer-to-Peer Energy Trading: Market Structure, Operational Layers, Energy Cooperatives and Multi-energy Systems,” *IET Renewable Power Generation,* vol. 19, no. 1, p. e70075, 2025.

[3] Z. Li and T. Ma, “Distributed photovoltaics with peer-to-peer electricity trading,” *Energy and Built Environment,* vol. 3, no. 4, pp. 424-432, 2022.

[4] P. Lusis and K. R. Khalilpour, “Short-term residential load forecasting: Impact of calendar effects and forecast granularity,” *Applied Energy,* vol. 205, pp. 654-669, 2017.

[5] W. Kong, Z. Y. Dong, Y. Jia, D. J. Hill, Y. Xu and Y. Zhang, “Short-term residential load forecasting based on LSTM recurrent neural network,” *IEEE Transactions on Smart Grid,* vol. 10, no. 1, pp. 841-851, 2019.

[6] E. McKenna, I. Richardson and M. Thomson, “Smart meter data: Balancing consumer privacy concerns with legitimate applications,” *Energy Policy,* vol. 41, pp. 807-814, 2012.

[7] J. D. Fernández, S. P. Menci, C. M. Lee, A. Rieger and G. Fridgen, “Privacy-preserving federated learning for residential short-term load forecasting,” *Applied Energy,* vol. 326, 2022.

[8] J. Evans, “Major_project_final,” 28 April 2026. [Online]. Available: https://github.com/JakeEvans6179/Major_project_final. [Accessed 26 April 2026].

[9] UK Power Networks, “SmartMeter Energy Consumption Data in London Households,” 2014. [Online]. Available: https://data.london.gov.uk/dataset/smartmeter-energy-consumption-data-in-london-households-vqm0d/. [Accessed 26 April 2026].

[10] Met Office, “MIDAS Open: UK hourly weather observation data, v202507,” 2025. [Online]. Available: https://data.ceda.ac.uk/badc/ukmo-midas-open/data/uk-radiation-obs/dataset-version-202507/greater-london/00708_heathrow/qc-version-1. [Accessed 26 April 2026].

[11] C. Briggs, Z. Fan and P. Andras, “Federated Learning for Short-Term Residential Load Forecasting,” *IEEE Open Access Journal of Power and Energy,* vol. 9, pp. 573-583, 2022.

[12] D. Chen, S. Barker, A. Subbaswamy, D. Irwin and P. Shenoy, “Non-Intrusive Occupancy Monitoring using Smart Meters,” in *Proceedings of the 5th ACM Workshop on Embedded Systems For Energy-Efficient Buildings*, Rome, 2013.

[13] L. Zhu, Z. Liu and S. Han, “Deep leakage from gradients,” in *Advances in Neural Information Processing*

3] *Systems*, 2019.

[1 J. Geiping, H. Bauermeister, H. Dröge and M. Moeller, "Inverting gradients - How easy is it to break
4] privacy in federated learning?," in *Advances in Neural Information Processing Systems*, 2020.

[1 Y. He, F. Luo, G. Ranzi and W. Wong, "Short-Term Residential Load Forecasting Based on Federated
5] Learning and Load Clustering," in *IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, Aachen, 2021.

[1 W. Toussaint and D. Moodley, "Clustering residential electricity consumption data to create archetypes
6] that capture household behaviour in South Africa," *South African Computer Journal,* vol. 32, no. 2, pp. 1-34, 2020.

[1 A. D., "Energy management of small-scale PV-battery systems: A systematic review considering practical
7] implementation, computational requirements, quality of input data and battery degradation," *Renewable and Sustainable Energy Reviews,* pp. 555-570, 2019.

[1 IEA, "Approximately 100 million households rely on rooftop solar PV by 2030 – Analysis - IEA," 1
8] September 2022. [Online]. Available: https://www.iea.org/reports/approximately-100-million-households-rely-on-rooftop-solar-pv-by-2030.

[1 N. G. Chatzigeorgiou, "A review on battery energy storage systems: Applications, developments, and
9] research trends of hybrid installations in the end-user sector," *Journal of Energy Storage,* p. 111192, 2024.

[2 S. Sizer, "Solar PV Sizer," 2026. [Online]. Available: https://www.solarpvsizer.com/.
0]

[2 W. Zhou, "Review on the Battery Model and SOC Estimation Method," *Processes,* p. 1685, September
1] 2021.

[2 N. Orth, "Efficiency characterization of 26 residential photovoltaic battery storage systems," *Journal of*
2] *Energy Storage,* p. 107299, April 2023.

[2 J. Reid, "Renewable Energy Services in Wakefield | Sheffield Renewable Services," 28 November 2025.
3] [Online]. Available: https://sheffieldrenewables.com/solar-panel-efficiency/.

[2 G. Huang, "GitHub - GeorgeHuang12/Households with predictions," 2025. [Online]. Available:
4] https://github.com/GeorgeHuang12/household-with-prediction.

[2 F. Alfaverh, M. Denai and Y. Sun, "A Dynamic Peer-to-Peer Electricity Market Model for a Community
5] Microgrid With Price-Based Demand Response," *IEEE Transactions on Smart Grid,* pp. 3976-3991, 2023.

[2 S. Liaquat, T. Hussain, B. Celik, R. Fourney and T. M. Hansen, "Day-ahead continuous double auction-based peer-to-peer energy trading platform incorporating trading losses and network utilisation fee,"

6] *IET Smart Grid,* pp. 312-329, 2023.

[2 J. Hemelhof, M. Mihaylov and A. Nowé, “Improving Zero-Intelligence Plus for Call Markets,” 2019.
7]

[2 P. Angaphiwatchawal, P. Phisuthsaingam and S. Chaitusaney, “A k-Factor Continuous Double Auction-
8] Based Pricing Mechanism for the P2P Energy Trading in a LV Distribution System,” in *2020 17th International Conference on Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI-CON)*, Phuket, Thailand, 2020.

[2 P. Shrivastava, N. Hubballi and H. Agrawal, “Implication of Cyber Attacks in Smart-Grid Auction: A
9] Performance Evaluation Case Study,” in *2024 IEEE Middle East Conference on Communications and Networking (MECOM)*, Abu Dhabi, United Arab Emirates, 2024.

[3 P. Vytelingum, D. Cliff and N. R. Jennings, “Strategic bidding in continuous double auctions,” *Artificial*
0] *Intelligence,* vol. 172, no. 14, pp. 1700-1729, 2008.

[3 T. Couture and Y. Gagnon, “An analysis of feed-in tariff remuneration models: Implications for
1] renewable energy investment,” *Energy Policy,* vol. 38, no. 2, pp. 955-965, 2010.

[3 Y.-C. Hung and G. Michailidis, “Modeling and Optimization of Time-of-Use Electricity Pricing Systems,”
2] *IEEE Transactions on Smart Grid,* vol. 10, no. 4, pp. 4116-4127, 2019.

[3 C. Long, J. Wu, Y. Zhou and N. Jenkins, “Peer-to-peer energy trading in a community microgrid,” *2017*
3] *IEEE Power & Energy Society General Meeting,* pp. 1-5, 2017.

[3 P. Goncalves Da Silva, D. Ilic and S. Karnouskos, “The Impact of Smart Grid Prosumer Grouping on
4] Forecasting Accuracy and Its Benefits for Local Electricity Market Trading,” *IEEE Transactions on Smart Grid,* vol. 5, no. 1, pp. 402-410, 2014.

[3 A. Roy, J.-C. Olivier, F. Auger, B. Auvity, S. Bourguet and E. Schaeffer, “A comparison of energy allocation
5] rules for a collective self-consumption operation in an industrial multi-energy microgrid,” *Journal of Cleaner Production,* vol. 389, p. 136001, 2023.

[3 A. G. Singh and R. K. Chauhan, “An algorithmic-based energy sharing model with peer-to-peer energy
6] trading for clustered residential buildings,” *Journal of Building Engineering,* vol. 76, p. 106960, 2023.

[3 Dockdocs, “What is Docker?,” [Online]. Available: (https://docs.docker.com/get-started/docker-
7] overview/, n.d.).

[3 D. Doc, “Explanations of Docker compose up,” [Online]. Available:
8] https://docs.docker.com/reference/cli/docker/compose/up/#:~:text=Builds%2C%20(re)creates%2C,compose%20logs%20%2D%2Dfollow%20does)..

[3 J. Lan, “Major_project,” Github, 28 April 2026. [Online]. Available:

9] https://github.com/JiancongLan/Major_project. [Accessed 28 April 2026].

[40] A. Paudel, L. P. M. I. Sampath, J. Yang and H. B. Gooi, "Peer-to-Peer Energy Trading in Smart Grid Considering Power Losses and Network Fees," *IEEE Transactions on Smart Grid,* vol. 11, no. 6, pp. 4727-4737, 2020.

[41] G. Belgioioso, W. Ananduta, S. Grammatico and C. Ocampo-Martinez, "Operationally-Safe Peer-to-Peer Energy Trading in Distribution Grids: A Game-Theoretic Market-Clearing Mechanism," *IEEE Transactions on Smart Grid,* vol. 13, no. 4, pp. 2897-2907, 2022.

[42] G. Vieira and J. Zhang, "Peer-to-peer energy trading in a microgrid leveraged by smart contracts," *Renewable and Sustainable Energy Reviews,* vol. 143, p. 110900, 2021.

[43] "https://docs.docker.com/get-started/docker-overview/," [Online].

[44] T. M. Zhenpeng Li, "Peer-to-peer electricity trading in grid-connected residential communities with household distributed photovoltaic," *Applied Energy,* vol. 278, p. 115670, 2020.

[46] F. Alfaverh, M. Denai and Y. Sun, "A Dynamic Peer-to-Peer Electricity Market Model for a Community Microgrid With Price-Based Demand Response," *IEEE Transactions on Smart Grid,* vol. 14, no. 5, pp. 3976-3991, 2023.

[47] F. Alfaverh, M. Denai and Y. Sun, "A Dynamic Peer-to-Peer Electricity Market Model for a Community Microgrid With Price-Based Demand Response," *IEEE Transactions on Smart Grid,* vol. 14, no. 5, pp. 3976-3991, 9 2023.

[48] F. Alfaverh, M. Denai and Y. Sun, "A Dynamic Peer-to-Peer Electricity Market Model for a Community Microgrid With Price-Based Demand Response," *IEEE Transactions on Smart Grid,* pp. 3976-3991, 2023.

[49] S. Liaquat, T. Hussain, B. Celik, R. Fourney and T. M. Hansen, "Day-ahead continuous double auction-based peer-to-peer energy trading platform incorporating trading losses and network utilisation fee," *IET Smart Grid,* vol. 6, no. 3, pp. 312-329, 2023.

[50] F. Alfaverh, M. Denai and Y. Sun, "A Dynamic Peer-to-Peer Electricity Market Model for a Community Microgrid With Price-Based Demand Response," *IEEE Transactions on Smart Grid,* vol. 14, no. 5, pp. 3976-3991, 2023.

[51] T. Ma and Z. Li, "Peer-to-peer electricity trading in grid-connected residential communities with household distributed photovoltaic," *Applied Energy,* p. 115670, 2020.

[52] J. L. Github. [Online]. Available: https://github.com/JiancongLan/Major_project.git.